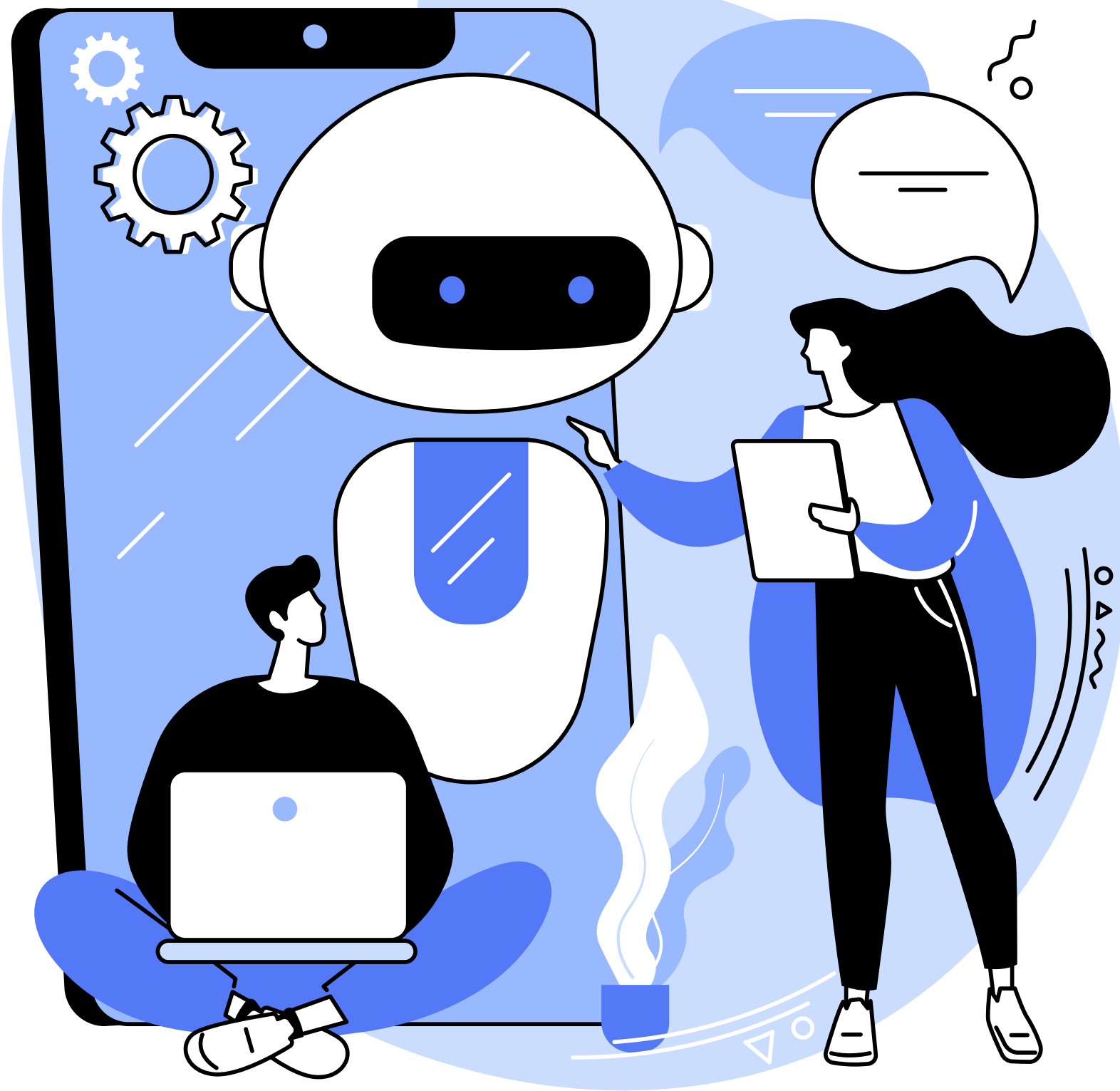




MobileNet CUDA

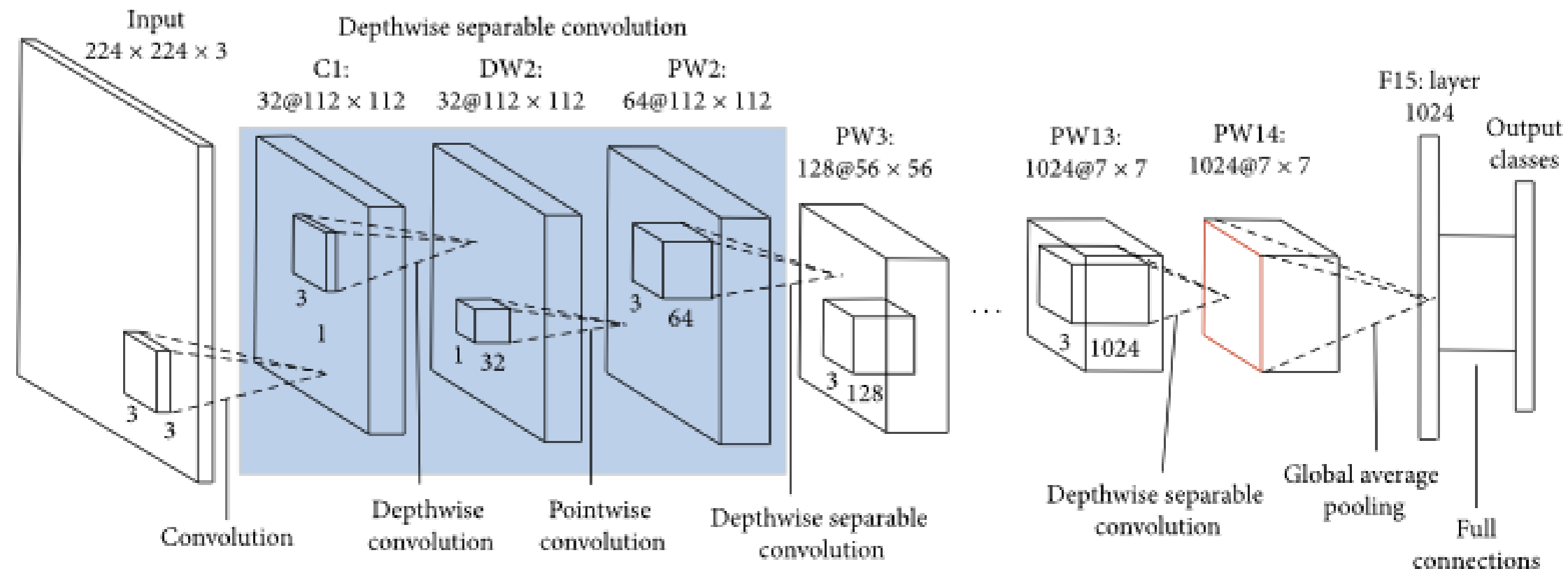
MobileNet

Depthwise Convolution

- Her filtre sadece tek bir kanalda çalışır.
- Normal conv gibi tüm kanalları tek seferde işlemiyor.
- Hesaplama maliyetini büyük ölçüde düşürür.

Pointwise Convolution (1×1 conv)

- Depthwise'tan gelen çıktıların kanallarını birleştirir.
- Asıl “öğrenme” gücü burada gerçekleşir.



MobileNetV2

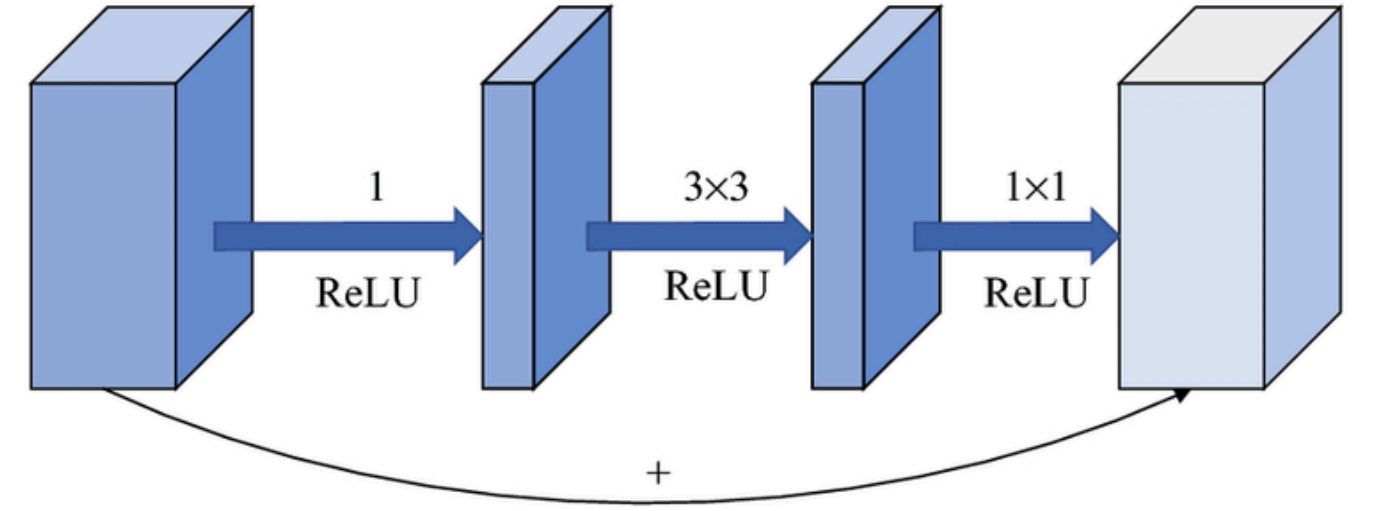
Inverted Residuals (Ters Çevrilmiş Artık Bloklar)

Normal residual bloklarda geniş (çok kanallı) katmanlar birbirine bağlanır.

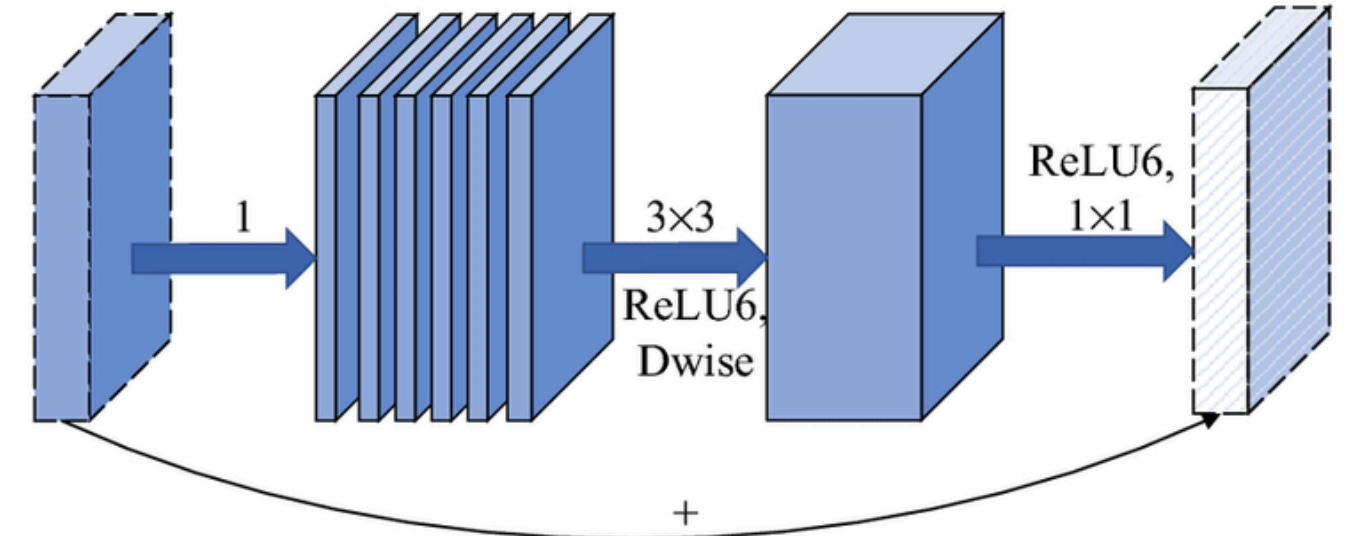
MobileNetV2'de ise tam tersi yapılır: Dar bir katman → genişletilir → işlenir → tekrar dar bir katmana döner.

Bu sayede hesaplama azalır, bilgi akışı hızlanır.

Daralt → Convolution → Genişlet



(a) Residual block



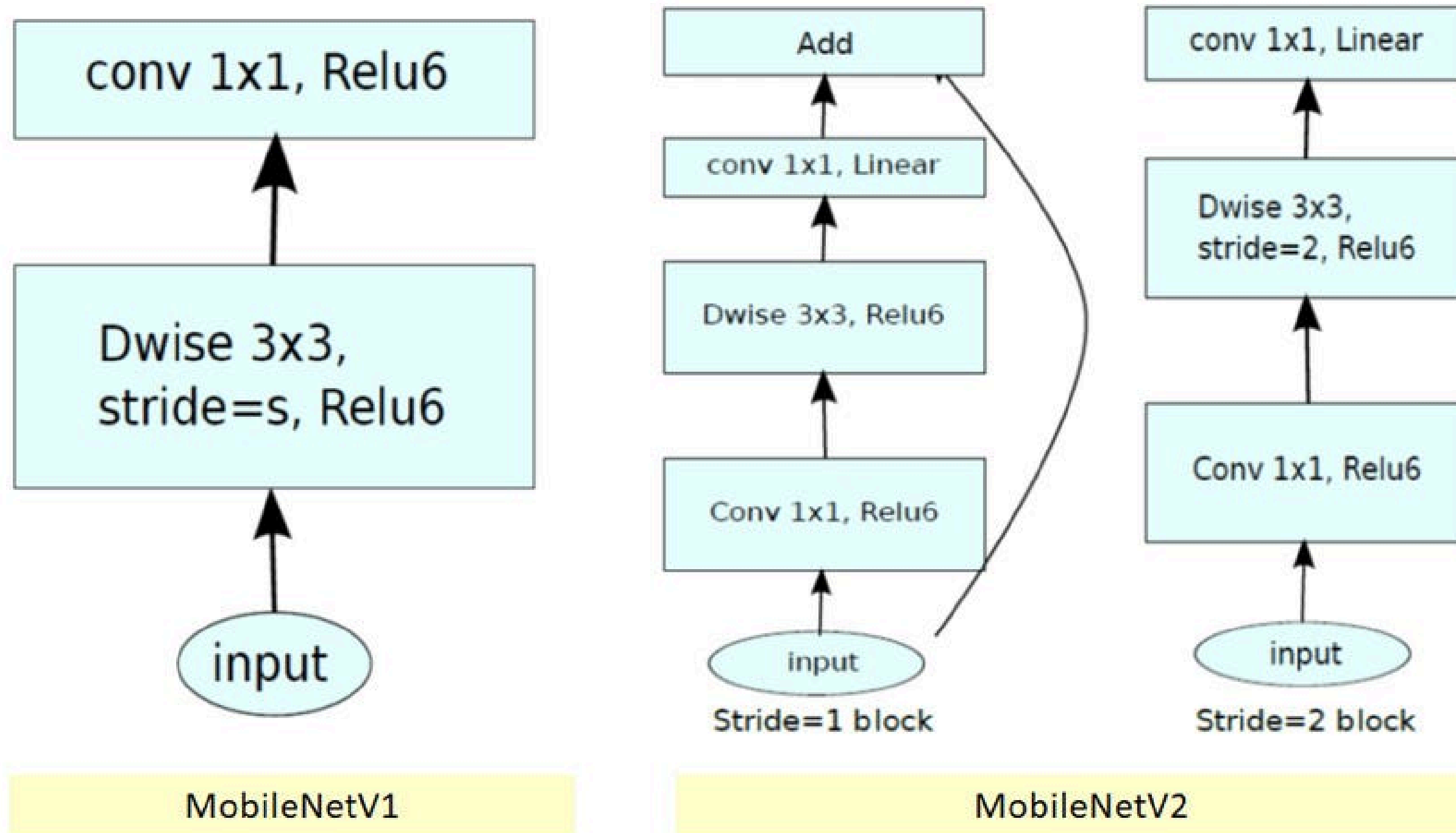
(b) Inverted residual block

Linear Bottlenecks (Lineer Dar Boğazlar)

Ara katmanlarda aktivasyon kullanılmaz (ReLU yok).

Bunun sebebi: Düşük boyutlu temsillerde ReLU bilgiyi yok edebiliyor.

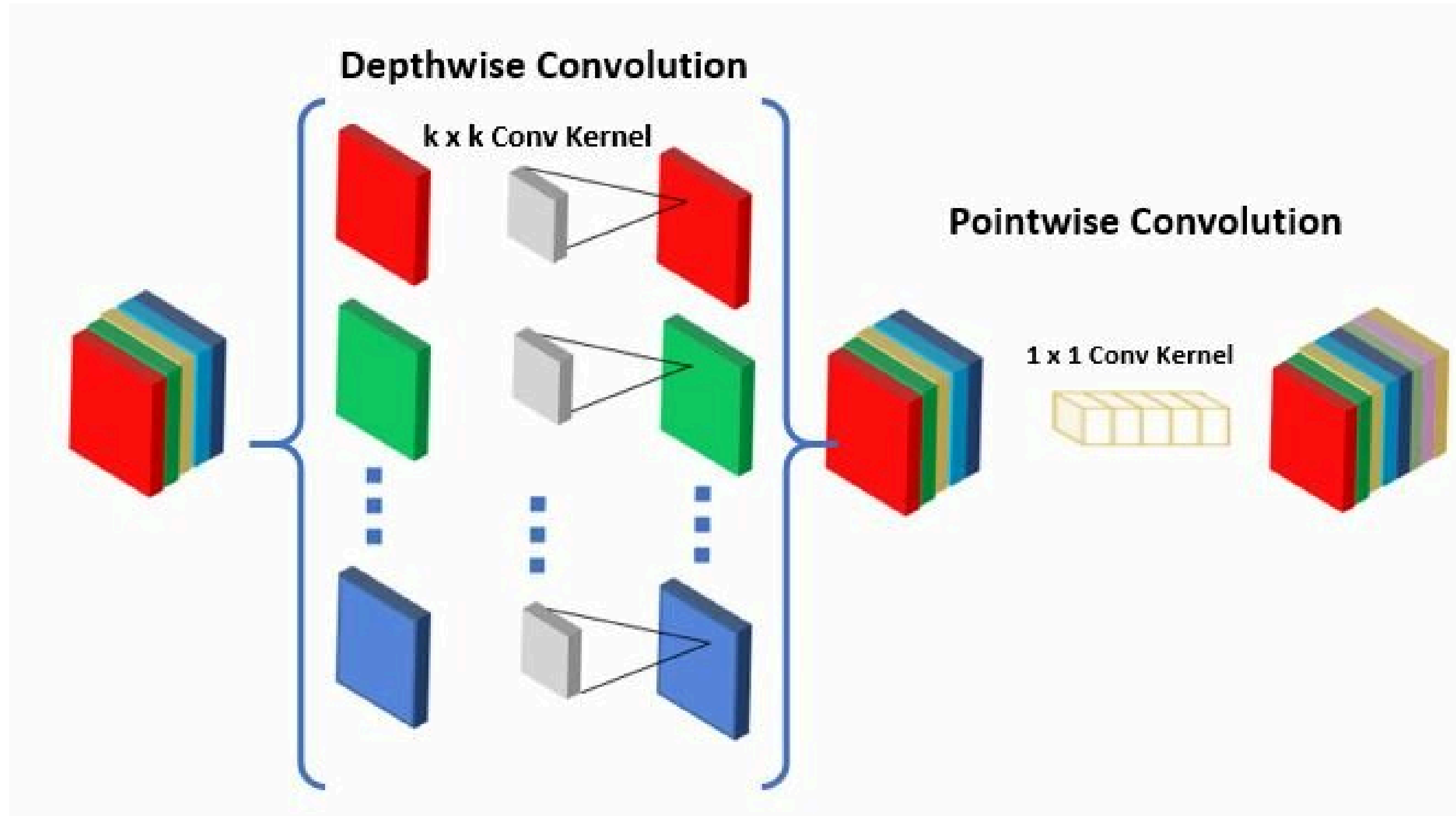
Lineer (aktifleşmesiz) katmanlar sayesinde bilgi kaybı azaltılır, doğruluk artar.



Depthwise Separable Convolutions (Ayrık Derinliksel Konvolüsyonlar)

- Depthwise: Her kanal kendi filtresi ile ayrı ayrı işlenir
- Pointwise (1×1): Kanallar karıştırılır

Bu yaklaşım normal konvolüsyona göre 8–9 kat daha az hesaplama gerektirir.

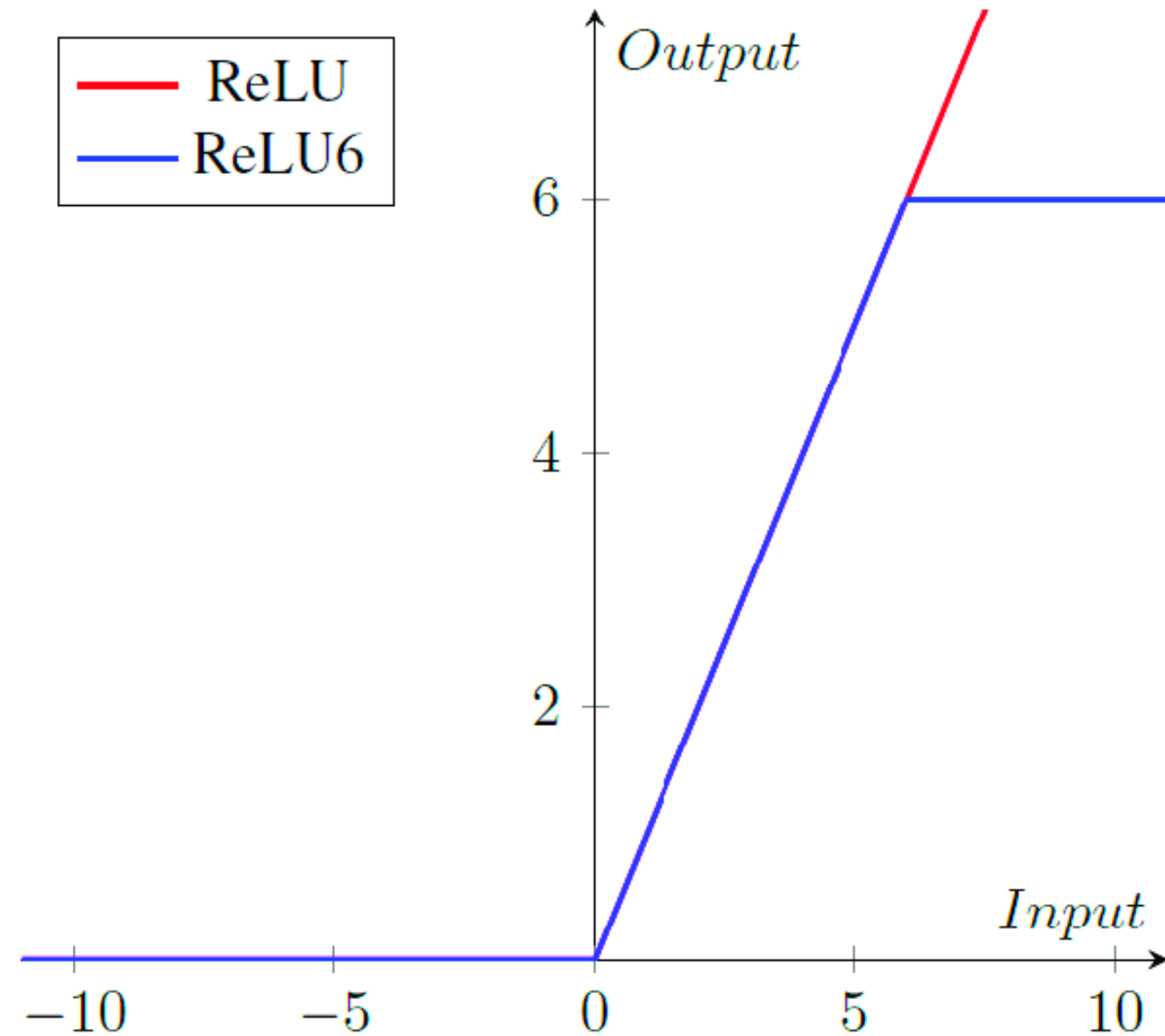


ReLU6 Aktivasyon Fonksiyonu

ReLU yerine ReLU6 kullanılır → çıktı değerlerini 0 ile 6 arasında sınırlar.

Bu, düşük hassasiyetli mobil işlemcilerde uygun bir yapıdır:

- Sayısal kararlılığı artırır
- Donanımda daha hızlı çalışır



MobileNet V2 Mimarisi

Başlangıç Katmanları

Input Layer

Modele verilen görüntü boyutu: 224×224 RGB.

İlk Konvolüsyon (Conv2D)

Stride = 2 ile uygulanan bir standart 3×3 konvolüsyon.

Görüntüyü yarı boyuta indirir (224→112) ve 32 kanal üretir.

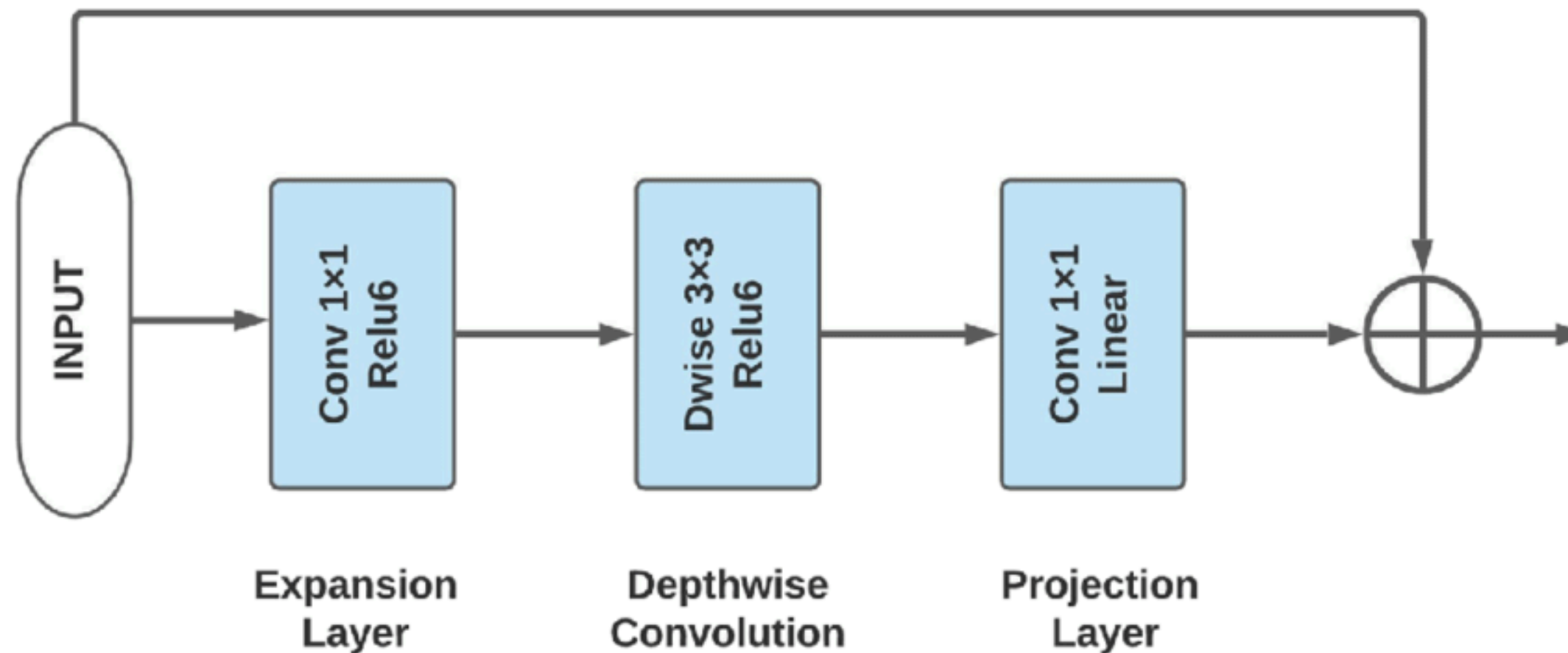
Input Size	Operator	Expansion Factor (t)	Output Channels (c)	Number of Repeats (n)	Stride (s)
224x224x3	Conv2D	-	32	1	2
112x112x32	Bottleneck	1	16	1	1

Inverted Residual Blokları (Ters Residual Blokları)

Her blok 3 parçadan oluşur:

Expansion Layer – Genişletme Katmanı

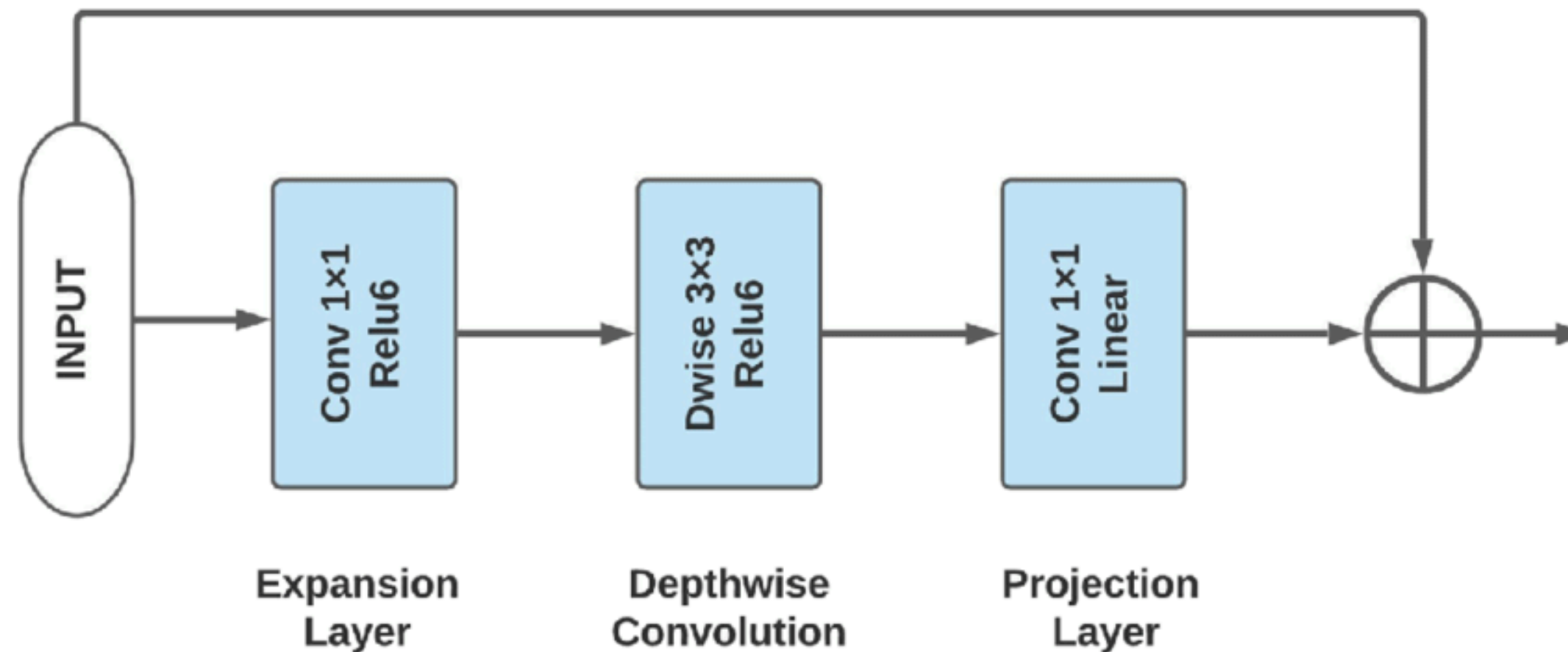
- 1×1 konvolüsyon ile kanal sayısını genişletir.
- Çoğu zaman 6 kat genişletir (expansion factor = 6).
- Aktivasyon: ReLU6



Inverted Residual Blokları (Ters Residual Blokları)

Depthwise Convolution – Kanal Bazlı Konvolüsyon

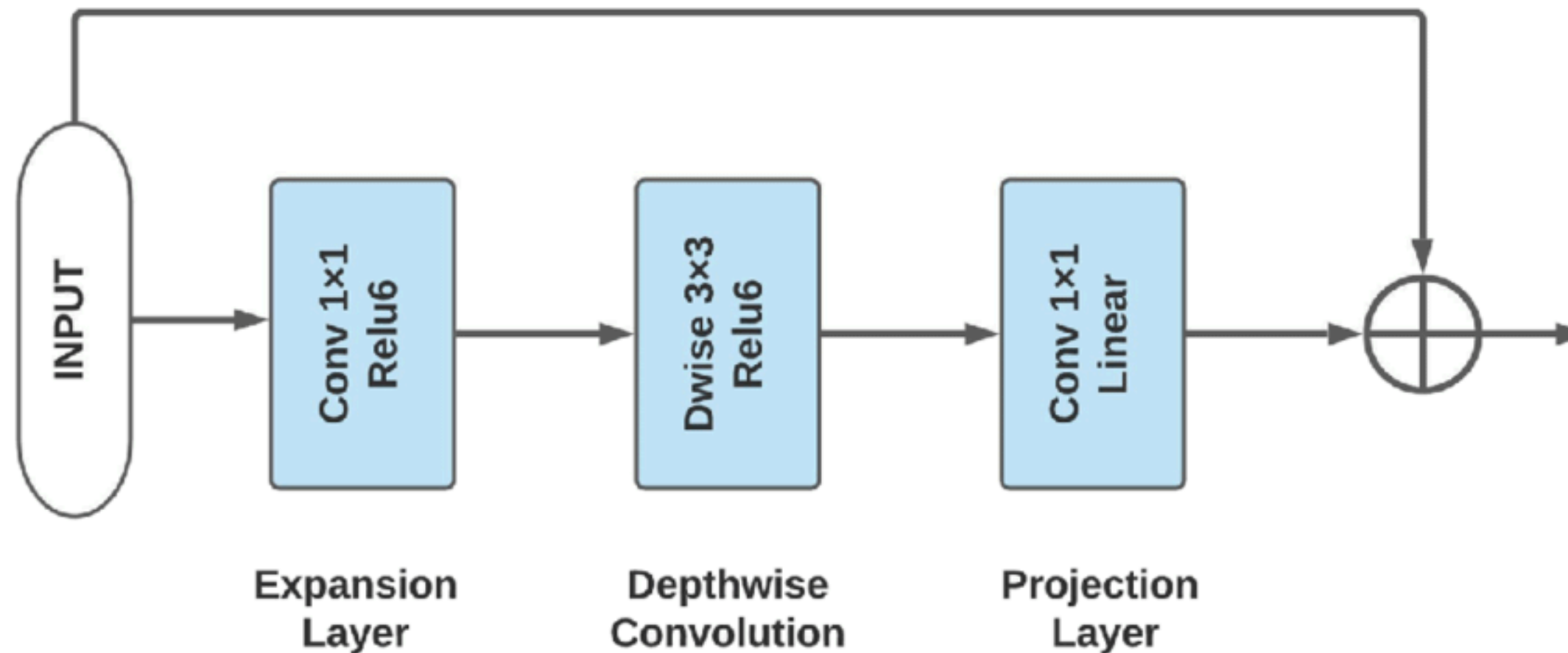
- Her kanal kendi filtreleri ile ayrı ayrı konvolüsyon yapılır.
- Çok düşük maliyetlidir.
- Aktivasyon: ReLU6



Inverted Residual Blokları (Ters Residual Blokları)

Projection Layer – Lineer Daraltma Katmanı

- 1×1 konvolüsyon ile kanallar tekrar daraltılır.
- Aktivasyon yok (linear) $\rightarrow$ bilgi kaybını azaltır (Linear Bottleneck).



Shortcut (Skip) Bağlantısı

Eğer giriş ve çıkış kanal sayıları eşitse kısayol bağlantısı eklenir.
Bu bağlantı derinliği artırmadan gradyan akışını iyileştirir.

Inverted Residual Bloklarının Detayları

İlk Blok

Expansion factor = 1 (genişletme yok)

Çıkış kanal sayısı = 16

Stride = 1

Sonraki Bloklar

Bir kısmı stride = 2 kullanarak boyutu küçültür (downsampling).

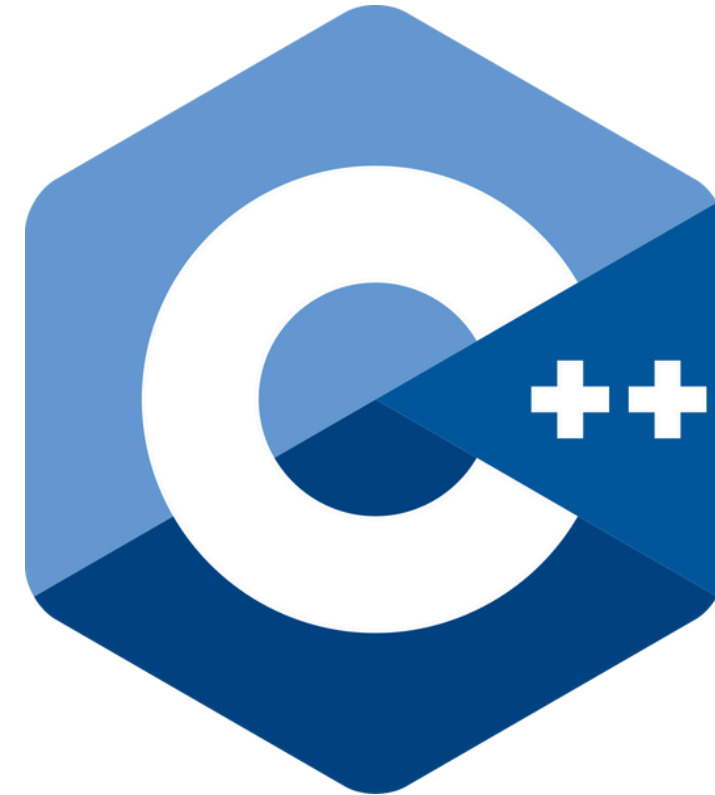
Expansion factor genelde = 6

Output kanal sayıları: 24, 32, 64, 96, 160, 320

Input Size	Operator	Expansion Factor (t)	Output Channels (c)	Number of Repeats (n)	Stride (s)
224x224x3	Conv2D	-	32	1	2
112x112x32	Bottleneck	1	16	1	1
112x112x16	Bottleneck	6	24	2	2
56x56x24	Bottleneck	6	32	3	2
28x28x32	Bottleneck	6	64	4	2
14x14x64	Bottleneck	6	96	3	1
14x14x96	Bottleneck	6	160	3	2
7x7x160	Bottleneck	6	320	1	1
7x7x320	Conv2D	-	1280	1	1
7x7x1280	AvgPool 7x7	-	-	1	-
1x1x1280	Conv2D	-	k (number of classes)	1	-

Amaç

- PyTorch'da hazır (pretrained) olan MobileNetV2 modelini C++ + CUDA tarafında yeniden kurmak.



PyTorch Modeli

 README  Apache-2.0 license



A PyTorch implementation of MobileNetV2

This is a PyTorch implementation of MobileNetV2 architecture as described in the paper [Inverted Residuals and Linear Bottlenecks: Mobile Networks for Classification, Detection and Segmentation](#).

<https://github.com/tonylins/pytorch-mobilenet-v2/tree/master?tab=readme-ov-file>

Giriş kanal sayısı, çıkış kanal sayısı
ve stride değeri alır

```
def conv_bn(input, output, stride):  
    return nn.Sequential(  
        nn.Conv2d(input, output, 3, stride, 1, bias=True),  
        # nn.BatchNorm2d(output),  
        nn.ReLU6(inplace=True)  
    )
```

PyTorch'ta
nn.Sequential, verdiğimiz
katmanları sırayla
çalıştıran bir kutudur.

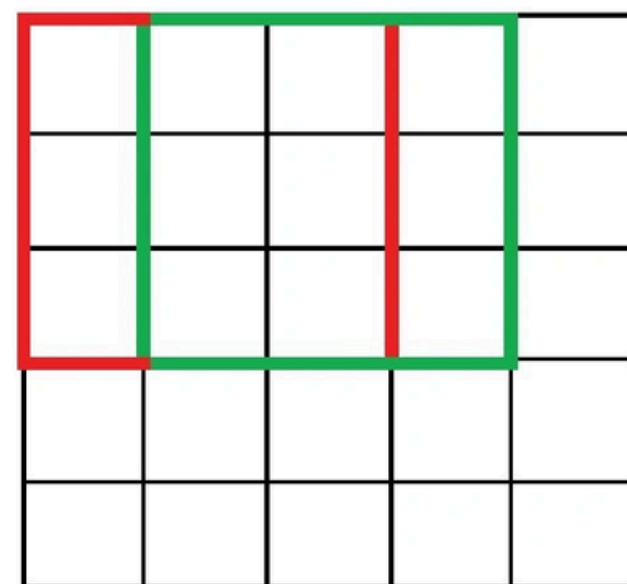
Bir 2D konvolüsyon yaratır.
Giriş, çıkış kanal sayısı,
kernel boyutu 3x3,
stride değeri,
padding = 1,

ReLU 6 bloğunu
oluşturur inplace =
true memory
kullanımını azaltır

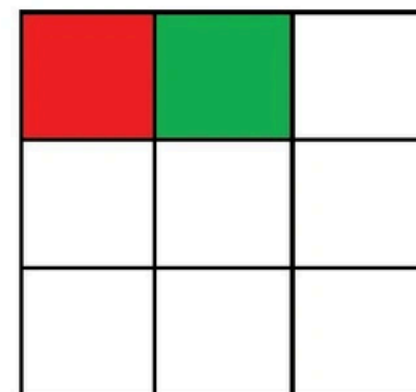
bias = True → ONNX versiyonunda batch norm kaldırıldığı için

Stride

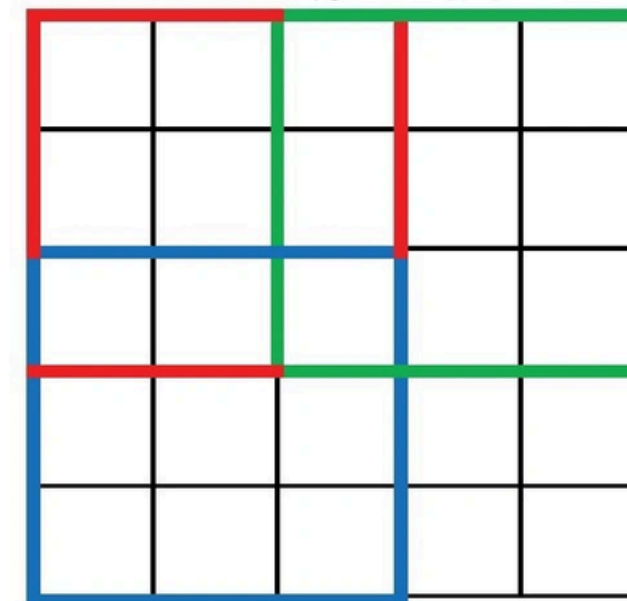
Convolution
with Stride=1



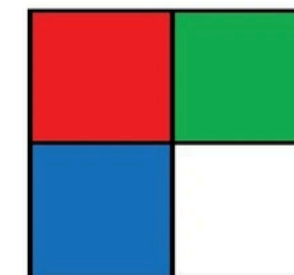
Output



Convolution
with Stride=2



Output

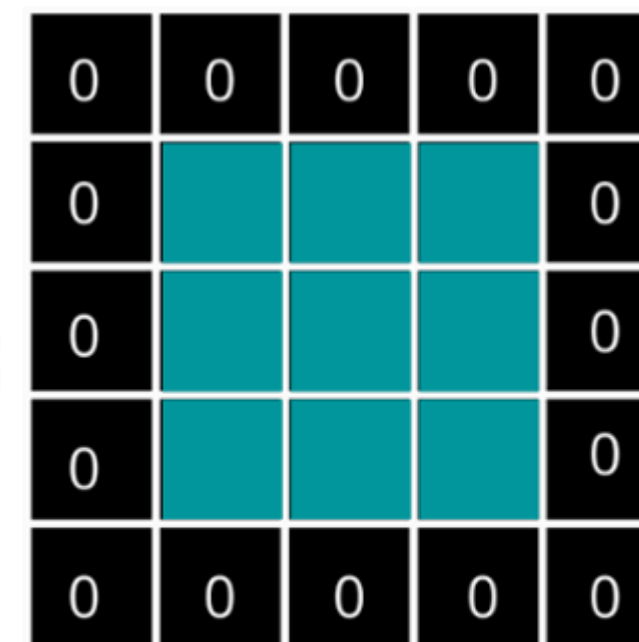


Padding



Input Image

Applying padding
of 1 on 3x3



Padded Image

Neden Batch Normalization Kaldırılmış?

$$y = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Bunu C++'da elle yapmak zor maliyetli ekstra kernel gerektiriyor ve inference aşamasında bile hesaplama overhead'i var

Kaldırınca

- konvolüsyon + bias yeterli olur
- kod basitleşir
- forward hesaplaması daha hafif olur
- kernel sayısı azalır

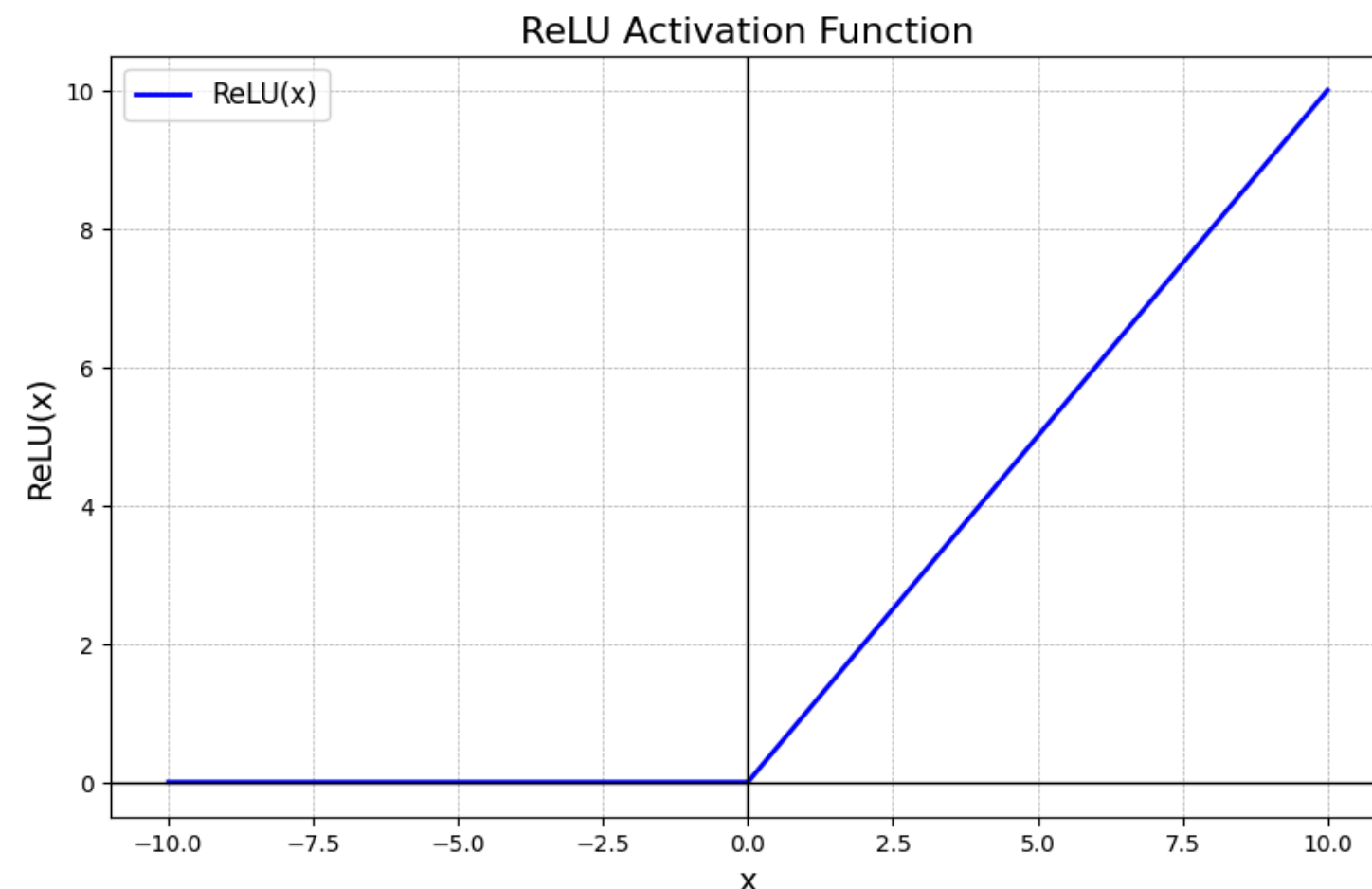
ReLU 6

$$\text{ReLU}(x) = \max(0, x)$$

- negatifler $\rightarrow 0$
- pozitifler $\rightarrow$ sınırsız büyüyebilir

$$\text{ReLU6}(x) = \min(\max(0, x), 6)$$

- negatifler $\rightarrow 0$
- pozitifler $\rightarrow$ en fazla 6



inplace memory

Normalde PyTorch

$$x \rightarrow \text{ReLU} \rightarrow y$$

ReLU yeni bir tensor oluşturur
x bellekte kalmaya devam eder
y başka bir bellek alanına yazılır

`inplace = True`

ReLU işleminden sonra yeni
tensor oluşturmaz,
x'in üzerini yazar.

$$x \rightarrow \text{ReLU6}(x) \rightarrow x$$

```
def conv_1x1_bn(input, output):  
    return nn.Sequential(  
        nn.Conv2d(input, output, 1, 1, 0, bias=True),  
        # nn.BatchNorm2d(output),  
        nn.ReLU6(inplace=True)  
    )
```

Bu fonksiyon bir 1×1 konvolüsyon + ReLU6 bloğu oluşturur.

Bir 1D konvolüsyon yaratır.

Giriş, çıkış kanal sayısı,
kernel boyutu 1x1,
stride değeri 1,
padding = 0,

Yine Batch norm
yok o yüzden
bias = True

Sonra ReLU6 bloğunu
oluşturur
inplace = True

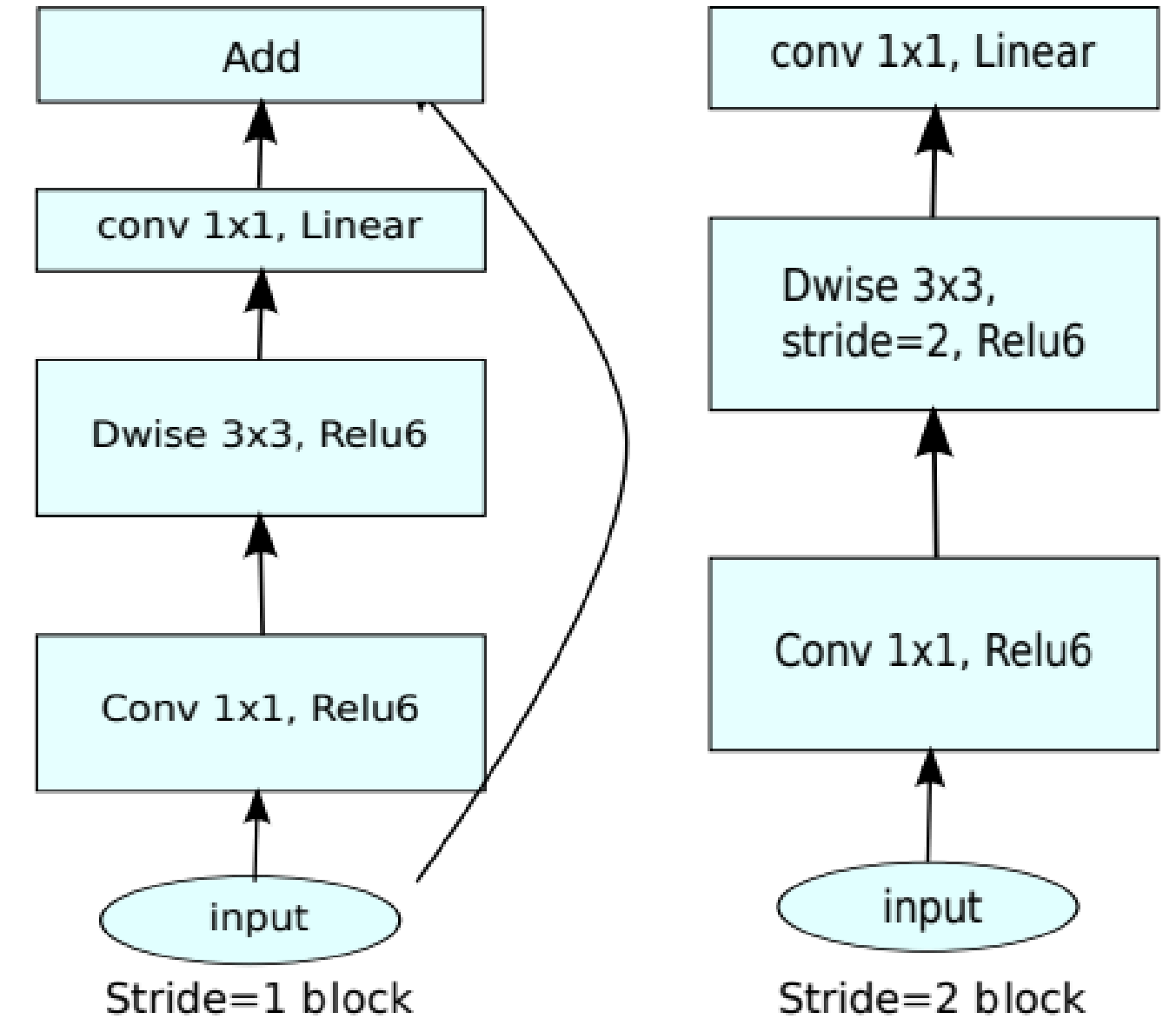
1×1 Conv'un görevi

Expansion ($t \times \text{input_channels}$ büyütme)

MobileNetV2'nin inverted residual bloklarında:

- önce 1×1 conv ile kanallar büyütülür (expand)
- sonra depthwise 3×3 yapılır
- sonra tekrar 1×1 conv ile küçültülür (project → linear)

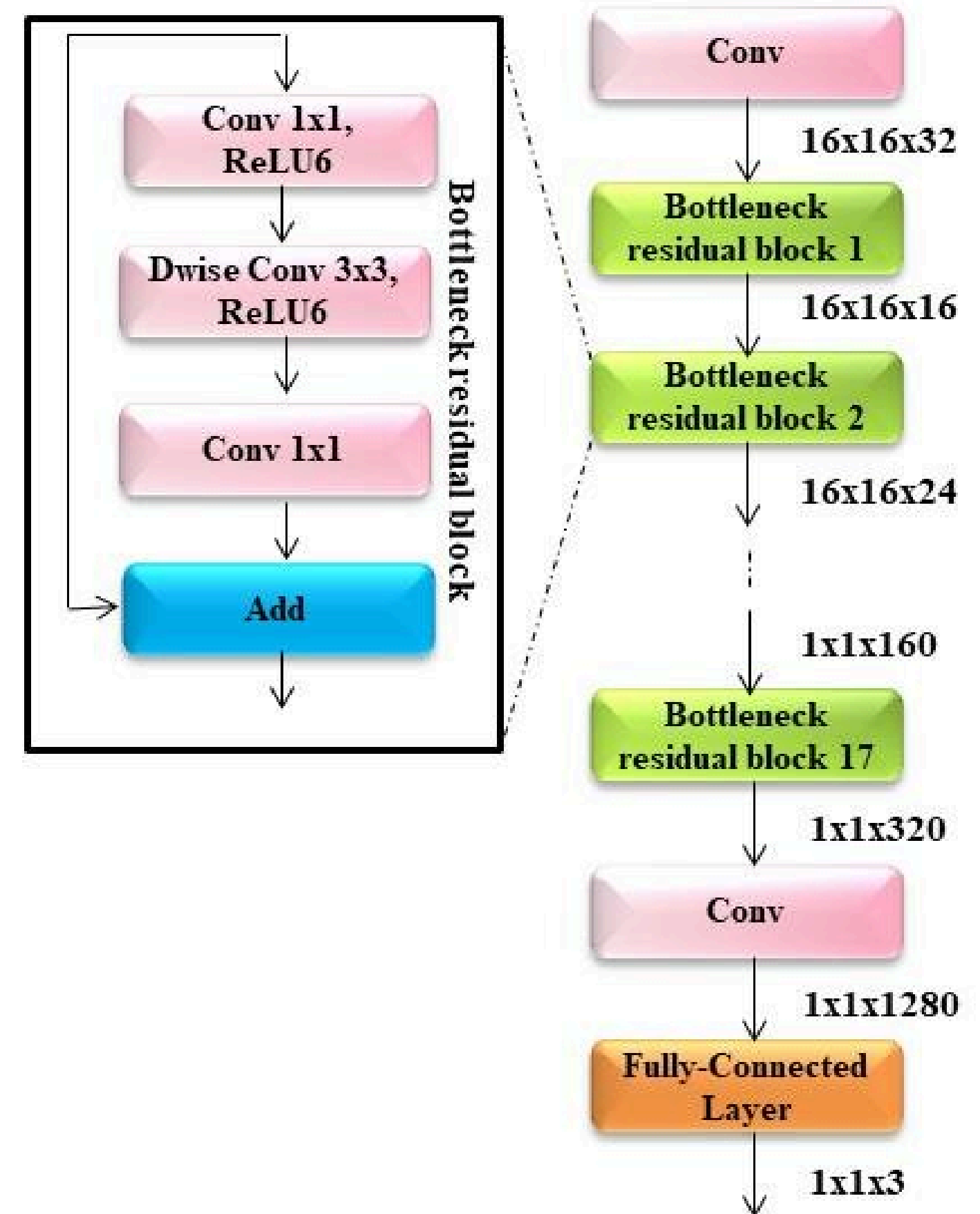
İşte bu ilk genişletme aşaması conv_1x1_bn.

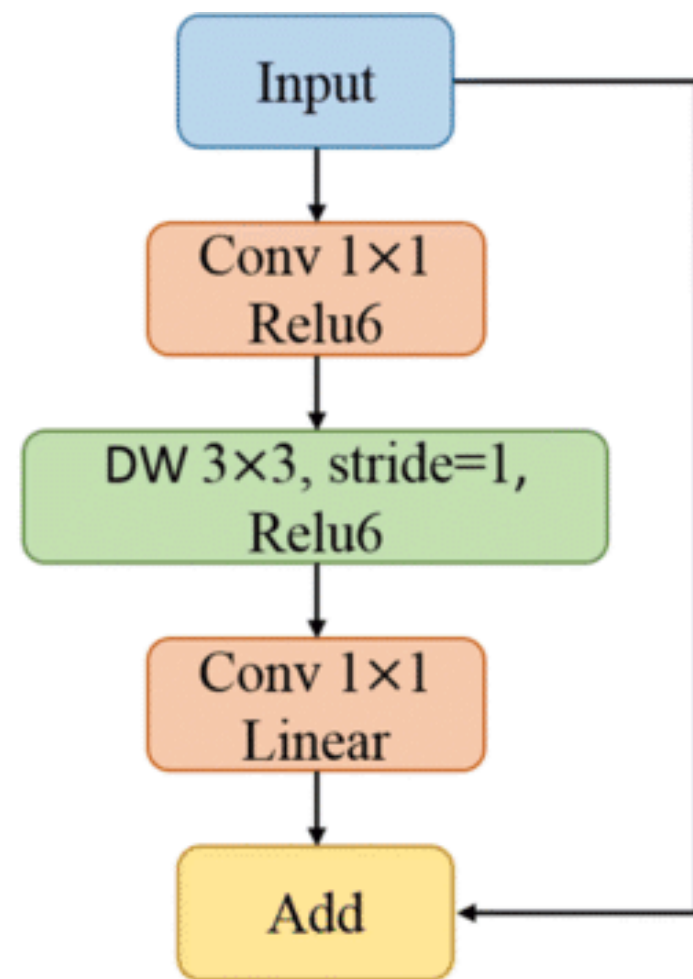


(d) Mobilenet V2

Son katmanda (son conv)
320 $\rightarrow$ 1280 dönüşümü
de 1 $\times$ 1 conv + ReLU6 ile yapılır.

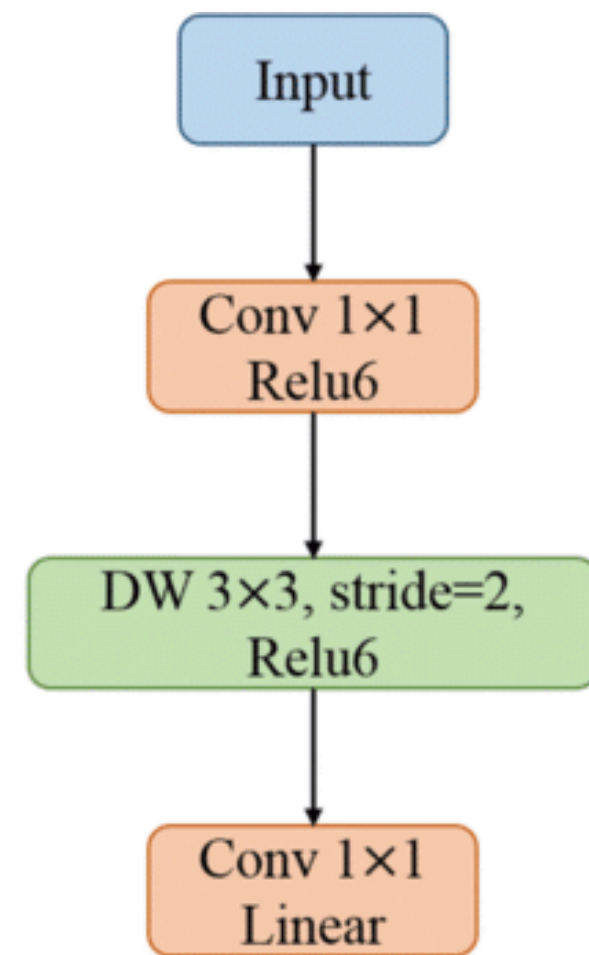
MobileNetV2,
sınıflandırmadan önce son 1 $\times$ 1
konvolüsyon ile kanalları
320'den 1280'e yükseltir; bu,
temsil gücünü artırır ve final
FC katmanının daha iyi
çalışmasını sağlar.





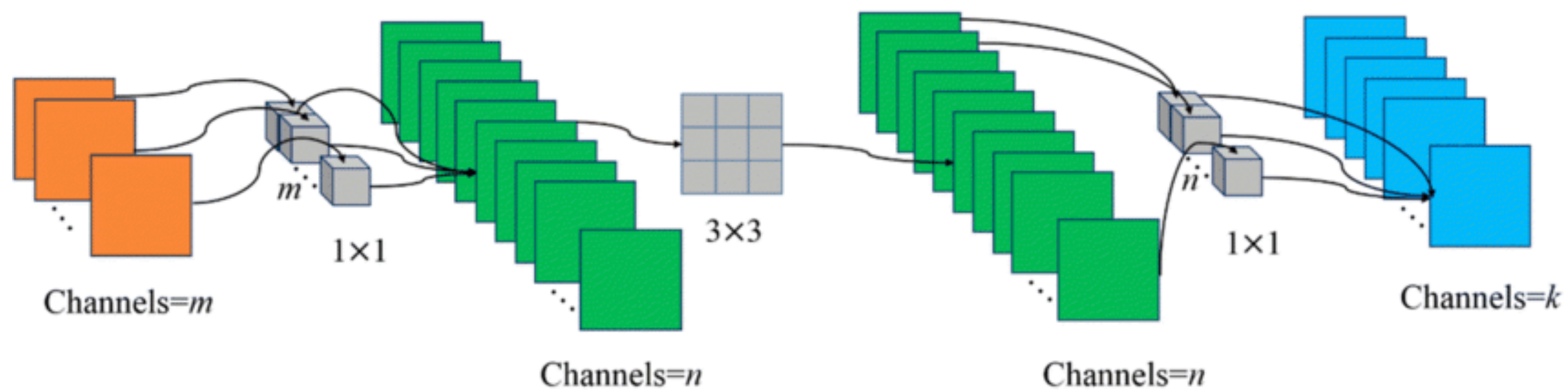
(a)

Filters= n



(b)

Filters= k



(c)

```

class InvertedResidual(nn.Module):
    def __init__(self, input, output, stride, expansion_ratio):
        super(InvertedResidual, self).__init__()
        self.residual_connect = stride == 1 and input == output
        hidden_dim = input * expansion_ratio

        if expansion_ratio == 1:
            self.conv = nn.Sequential(
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )
        else:
            self.conv = nn.Sequential(
                # pointwise convolution, ReLU6
                nn.Conv2d(input, hidden_dim, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )

```

Stride 1 ise ve `input == output` ise

O zaman residual bağlantı kullanılabilir.

Giriş tensörü ile çıkış tensörünü
toplama yapabilirsin
Çünkü boyutlar tam olarak aynı

Eğer `stride=2` olursa, feature map
küçülür → residual yapamaz.
Eğer kanal sayısı değişirse →
residual yapamaz.

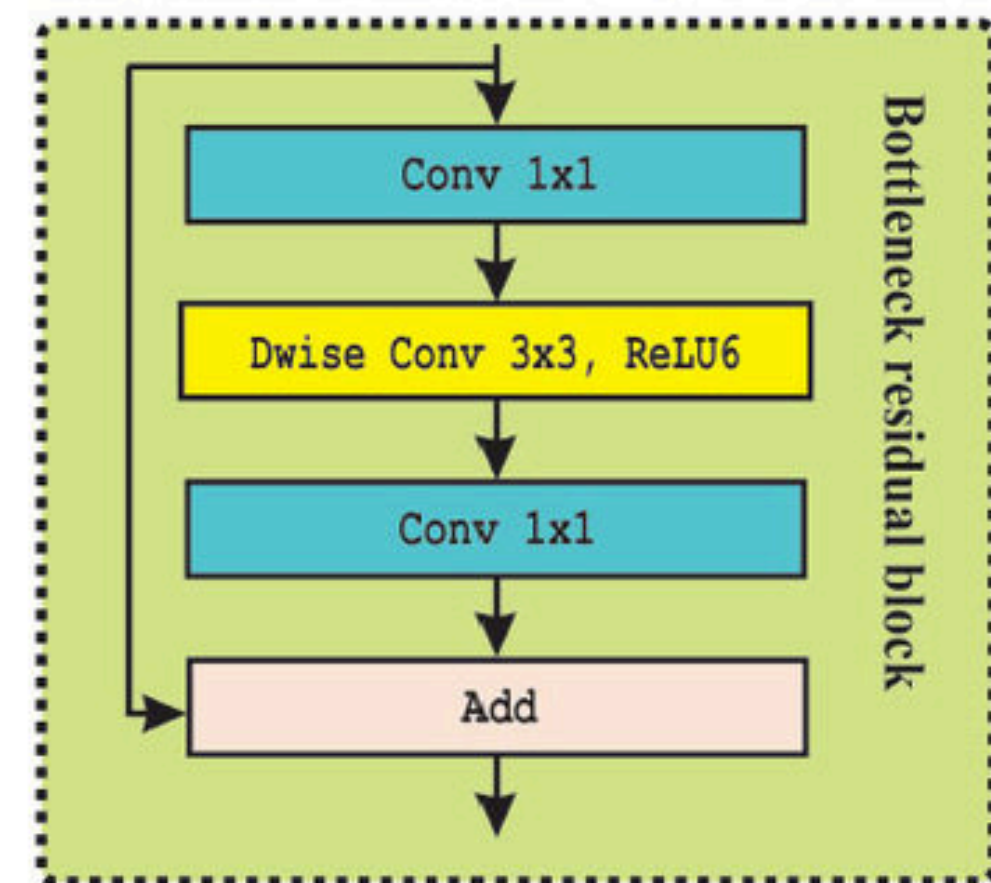
```

class InvertedResidual(nn.Module):
    def __init__(self, input, output, stride, expansion_ratio):
        super(InvertedResidual, self).__init__()
        self.residual_connect = stride == 1 and input == output
        hidden_dim = input * expansion_ratio

        if expansion_ratio == 1:
            self.conv = nn.Sequential(
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )
        else:
            self.conv = nn.Sequential(
                # pointwise convolution, ReLU6
                nn.Conv2d(input, hidden_dim, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )

```

MobileNetV2'de mantık şu:
 Blok önce kanalları genişletir
 Depthwise conv'u genişletilmiş
 kanala uygular
 Sonra tekrar küçültür
 Örnek:
 input = 24, expansion_ratio = 6 →
 hidden_dim = 144



```

class InvertedResidual(nn.Module):
    def __init__(self, input, output, stride, expansion_ratio):
        super(InvertedResidual, self).__init__()
        self.residual_connect = stride == 1 and input == output
        hidden_dim = input * expansion_ratio

        if expansion_ratio == 1:
            self.conv = nn.Sequential(
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )
        else:
            self.conv = nn.Sequential(
                # pointwise convolution, ReLU6
                nn.Conv2d(input, hidden_dim, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )

```

Expansion Ratio

With the exception of the first layer, we use constant expansion rate throughout the network. In our experiments we find that expansion rates between 5 and 10 result in nearly identical performance curves, with smaller networks being better off with slightly smaller expansion rates and larger networks having slightly better performance with larger expansion rates.

For all our main experiments we use expansion factor of 6 applied to the size of the input tensor. For example, for a bottleneck layer that takes 64-channel input tensor and produces a tensor with 128 channels, the intermediate expansion layer is then $64 \cdot 6 = 384$ channels.

From paper

expansion ratio $t = 6$ tamamen deneysel olarak en iyi accuracy-latency dengesini verdiği için seçilmiş.

```

class InvertedResidual(nn.Module):
    def __init__(self, input, output, stride, expansion_ratio):
        super(InvertedResidual, self).__init__()
        self.residual_connect = stride == 1 and input == output
        hidden_dim = input * expansion_ratio

        if expansion_ratio == 1:
            self.conv = nn.Sequential(
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )
        else:
            self.conv = nn.Sequential(
                # pointwise convolution, ReLU6
                nn.Conv2d(input, hidden_dim, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )

```

expansion_ratio ==1 -> özel bir durum.

Eğer genişletme yoksa (t=1)

O zaman blok sadece şunlardan oluşur:

Sadece Depthwise + Pointwise

İlk layerde paper t = 1 kullanıyor

Çünkü düşük seviyeli özelliklerde genişletme gerekli değil, hesaplama maliyeti gereksiz artar.

Stride=2 ile downsample eden ilk blok zaten pahalı, residual bağlantısı çalışmaz

```

class InvertedResidual(nn.Module):
    def __init__(self, input, output, stride, expansion_ratio):
        super(InvertedResidual, self).__init__()
        self.residual_connect = stride == 1 and input == output
        hidden_dim = input * expansion_ratio

        if expansion_ratio == 1:
            self.conv = nn.Sequential(
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )
        else:
            self.conv = nn.Sequential(
                # pointwise convolution, ReLU6
                nn.Conv2d(input, hidden_dim, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )

```

1. Aşama: Expand (1×1 Conv + ReLU6)

“Non-linearity yoksa model güçlü öğrenemez.
ReLU6 ile non-linearity uygulamak için tensörü geniş bir alana çıkarmalıyız.”

2. Aşama: Depthwise Convolution

her kanal kendine özel 3×3 filtre ile işleniyor.
normal conv'dan 8–9 kat daha ucuz

```

class InvertedResidual(nn.Module):
    def __init__(self, input, output, stride, expansion_ratio):
        super(InvertedResidual, self).__init__()
        self.residual_connect = stride == 1 and input == output
        hidden_dim = input * expansion_ratio

        if expansion_ratio == 1:
            self.conv = nn.Sequential(
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )
        else:
            self.conv = nn.Sequential(
                # pointwise convolution, ReLU6
                nn.Conv2d(input, hidden_dim, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # depthwise convolution
                nn.Conv2d(hidden_dim, hidden_dim, 3, stride,
                    [1, 1, 1], groups=hidden_dim, bias=True),
                # nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True),
                # pointwise convolution, linear
                nn.Conv2d(hidden_dim, output, 1, 1, 0, bias=True),
                # nn.BatchNorm2d(output),
            )

```

3. Aşama: Project

geniş alandan dar çıkışa
projeksiyon yapıyor:
hidden_dim (geniş) → output (dar)
bu katmanda ReLU yok

“Dar (bottleneck) alanda
non-linearity bilgi yok
ediyor.

Bu yüzden çıkış katmanı
LINEAR olmalı.”

```
def forward(self, x):  
    if self.residual_connect:  
        return x + self.conv(x)  
    else:  
        return self.conv(x)
```

Resudial connection varsa
x ile self.conv(x) aynı boyutta olduğundan toplanabilir.

self.conv

Bir katman zinciri (Conv → ReLU6 → DW Conv → ReLU6 → PW Conv)

self.conv(x)

x'i bu tüm katmanlardan geçir, sonucu döndür

```

class MobileNetV2(nn.Module):

    def __init__(self, num_class=1000, onnx_path='mobilenet_v2.onnx'):
        super(MobileNetV2, self).__init__()
        input_channel = 32
        last_channel = 1280
        inverted_residual_settings = [
            # t, c, n, s
            # t: expansion ratio
            # c: number of output channels
            # n: repeated times of identical layers
            # s: stride of first layer
            [1, 16, 1, 1],
            [6, 24, 2, 2],
            [6, 32, 3, 2],
            [6, 64, 4, 2],
            [6, 96, 3, 1],
            [6, 160, 3, 2],
            [6, 320, 1, 1],
        ]

```

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

```

# build first conv2d
self.features = [conv_bn(3, input_channel, 2)]

# build several inverted residuals
for expansion_ratio, output_channel, repeated_times, stride_first in inverted_residual_settings:
    for i in range(repeated_times):
        if i == 0:
            self.features.append(InvertedResidual(
                input_channel, output_channel, stride_first, expansion_ratio))
        else:
            self.features.append(InvertedResidual(
                input_channel, output_channel, 1, expansion_ratio))
        input_channel = output_channel

# build last several layers
self.features.append(conv_1x1_bn(input_channel, last_channel))

# make it nn.Sequential
self.features = nn.Sequential(*self.features)

# building classifier
self.classifier = nn.Linear(last_channel, num_class)

# initialize weights from given onnx file
self.initialize_weights(onnx_path)

```

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2

input = 3 (RGB)
 output =
 input_channel = 32
 stride = 2

$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1

```

# build first conv2d
self.features = [conv_bn(3, input_channel, 2)]

# build several inverted residuals
for expansion_ratio, output_channel, repeated_times, stride_first in inverted_residual_settings:
    for i in range(repeated_times):
        if i == 0:
            self.features.append(InvertedResidual(
                input_channel, output_channel, stride_first, expansion_ratio))
        else:
            self.features.append(InvertedResidual(
                input_channel, output_channel, 1, expansion_ratio))
        input_channel = output_channel

# build last several layers
self.features.append(conv_1x1_bn(input_channel, last_channel))

# make it nn.Sequential
self.features = nn.Sequential(*self.features)

# building classifier
self.classifier = nn.Linear(last_channel, num_class)

# initialize weights from given onnx file
self.initialize_weights(onnx_path)

```

Weights comes from ONNX

Bu listeyi sırayla
 çalışan
 tek bir modüle
 dönüştürür
 nn.Sequential

$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	conv2d 1x1	-	1280	1	1

1280 → 1000
 fully connected
 (ImageNet)
 classifier

```
def forward(self, x):  
    x = self.features(x)  
    x = x.mean(3).mean(2)  
    x = self.classifier(x)  
    return x
```

features, Conv + InvertedResidual + Conv1x1 bloklarının hepsini içeren tüm CNN feature extractor kısmıdır.

self.feature'dan çıkan $\rightarrow [B, C, H, W]$

Global Average Pooling (GAP)

$[B, C, H]$ $\text{mean}(3) \rightarrow \text{width (W) boyunca ortalama al}$

$[B, C]$ $\text{mean}(2) \rightarrow \text{height (H) boyunca ortalama al}$

$[B, C, H, W] \rightarrow [B, C]$

B = batch size

C = kanal sayısı (ör: 1280)

H = yükseklik

W = genişlik

GAP'tan gelen 1280 feature $\rightarrow$ 1000 sınıf skoruna dönüşür.

```

def initialize_weights(self, onnx_path):
    initializers = onnx.load(onnx_path).graph.initializer

    # classifier
    assert self.classifier.weight.data.shape == torch.tensor(
        to_array(initializers[0])).shape
    assert self.classifier.bias.data.shape == torch.tensor(
        to_array(initializers[1])).shape
    self.classifier.weight.data = torch.tensor(to_array(initializers[0]))
    self.classifier.bias.data = torch.tensor(to_array(initializers[1]))

    # first conv2d
    assert self.features[0][0].weight.data.shape == torch.tensor(
        to_array(initializers[2])).shape
    assert self.features[0][0].bias.data.shape == torch.tensor(
        to_array(initializers[3])).shape
    self.features[0][0].weight.data = torch.tensor(
        to_array(initializers[2]))
    self.features[0][0].bias.data = torch.tensor(to_array(initializers[3]))

    # inverted residuals
    idx_inv_res = range(1, len(self.features) - 1)
    idx_init = 4
    for i in idx_inv_res:
        assert isinstance(self.features[i], InvertedResidual)
        for j in range(len(self.features[i].conv)):
            if isinstance(self.features[i].conv[j], nn.Conv2d):
                assert self.features[i].conv[j].weight.data.shape == torch.tensor(
                    to_array(initializers[idx_init])).shape
                assert self.features[i].conv[j].bias.data.shape == torch.tensor(
                    to_array(initializers[idx_init + 1])).shape
                self.features[i].conv[j].weight.data = torch.tensor(
                    to_array(initializers[idx_init]))
                self.features[i].conv[j].bias.data = torch.tensor(
                    to_array(initializers[idx_init + 1]))
                idx_init += 2

```

önce ONNX modelini yüklüyor

Classifier (Linear layer) ağırlıklarını ONNX'ten alıyor

Önce shape kontrolü yapılıyor (garanti olsun diye).
Sonra PyTorch layer'ına kopyalanıyor.

İlk Conv2D Katmanı Ağırlıkları

features[0][0] = Conv2d

features[0][1] = ReLU6

Bu yüzden ilk Conv'un weight ve bias'ı
ONNX'ten okunuyor.

InvertedResidual Blok Ağırlıkları

Bu yapıda her Conv2D için sırayla:

initializer[idx_init] → weight

initializer[idx_init+1] → bias

ONNX Modeli

Resmi MobileNetV2

Imagenet üzerinde önceden eğitilmiş gerçek MobileNetV2 ağırlıkları içeriyor

Tamamen dondurulmuş ağırlıklar içeriyor

TensorFlow veya PyTorch fark etmez → ONNX formatı tüm frameworklerden bağımsızdır.

Bizim PyTorch MobileNetV2 modeli, bu ONNX'teki ağırlıkları birebir kullanıyor.

```
# last conv2d
assert self.features[len(self.features) - 1][0].weight.data.shape == torch.tensor(
    to_array(initializers[idx_init])).shape
assert self.features[len(self.features) - 1][0].bias.data.shape == torch.tensor(
    to_array(initializers[idx_init + 1])).shape
self.features[len(
    self.features) - 1][0].weight.data = torch.tensor(to_array(initializers[idx_init]))
self.features[len(
    self.features) - 1][0].bias.data = torch.tensor(to_array(initializers[idx_init + 1]))
idx_init += 2

assert idx_init == len(initializers) - 1
```

ONNX modelindeki SON 1×1 convolution katmanının
weight'ini
bias'ını

PyTorch modeline yükler.

Şekillerin birebir aynı olduğunu doğrular.

ONNX'teki tüm ağırlıkların eksiksiz tüketildiğini garanti eder.

```
def compare_inference_results(ref_onnx_path, my_onnx_path, rel_tol=1e-4):
    input_data = np.random.randn(1, 3, 244, 244).astype(np.float32)

    sess_ref = rt.InferenceSession(ref_onnx_path)
    input_name = sess_ref.get_inputs()[0].name
    output_name = sess_ref.get_outputs()[0].name
    pred_ref = sess_ref.run([output_name], {input_name: input_data})[0]

    sess_my = rt.InferenceSession(my_onnx_path)
    input_name = sess_my.get_inputs()[0].name
    output_name = sess_my.get_outputs()[0].name
    pred_my = sess_my.run([output_name], {input_name: input_data})[0]

    # comparing floating point numbers
    for val_ref, val_my in zip(pred_ref[0], pred_my[0]):
        assert(math.isclose(val_ref, val_my, rel_tol=rel_tol))
```

Rastgele bir giriş
oluştur -
1 adet görüntü
(batch=1)
3 kanal (RGB)
244×244 boyut
float32

rt.InferenceSession → ONNX Runtime ile
modeli yükler
get_inputs()[0].name → ONNX modelindeki
giriş tensorünün adı
sess_ref.run() → modeli çalıştırır
pred_ref → referans modelin çıktıları (1000
sınıflı Softmax öncesi logits)

Aynı input
MobileNetV2
ONNX'i için de
çalıştırılır.

“Eğer iki sayı
%0.01'den daha fazla
farklıysa hata ver.”

```
✓ if __name__ == '__main__':  
    net = MobileNetV2()  
    dummy_input = torch.randn(1, 3, 244, 244)  
✓ torch.onnx.export(net, dummy_input, "mobilenet_v2_my.onnx", input_names=[  
    | | | | | 'input.1'], output_names=['output.1'])  
    compare_inference_results('mobilenet_v2.onnx', 'mobilenet_v2_my.onnx')  
    summary(net, (3, 244, 244), batch_size=1, device='cpu')
```

1. PyTorch MobileNetV2 mimarisini doğru implemente ettik mi
Katman yapısı birebir eşleşti mi?
2. ONNX dosyasından ağırlıkları doğru okuduk mu ve yükledik mi?
3. Bizim modelin çıktısı orijinal MobileNetV2 ONNX ile birebir aynı mı?

```
(torch-old) bash-5.1$ python pytorchMobileNetV2.py
Model başarıyla oluşturuldu, ONNX export edildi ve karşılaştırma başarılı!
```

Layer (type)	Output Shape	Param #
Conv2d-1	[1, 32, 122, 122]	896
ReLU6-2	[1, 32, 122, 122]	0
Conv2d-3	[1, 32, 122, 122]	320
ReLU6-4	[1, 32, 122, 122]	0
Conv2d-5	[1, 16, 122, 122]	528
InvertedResidual-6	[1, 16, 122, 122]	0
Conv2d-7	[1, 96, 122, 122]	1,632
ReLU6-8	[1, 96, 122, 122]	0
Conv2d-9	[1, 96, 61, 61]	960
ReLU6-10	[1, 96, 61, 61]	0
Conv2d-11	[1, 24, 61, 61]	2,328
InvertedResidual-12	[1, 24, 61, 61]	0
Conv2d-13	[1, 144, 61, 61]	3,600
ReLU6-14	[1, 144, 61, 61]	0
Conv2d-15	[1, 144, 61, 61]	1,440
ReLU6-16	[1, 144, 61, 61]	0
Conv2d-17	[1, 24, 61, 61]	3,480
InvertedResidual-18	[1, 24, 61, 61]	0
Conv2d-19	[1, 144, 61, 61]	3,600
ReLU6-20	[1, 144, 61, 61]	0
Conv2d-21	[1, 144, 31, 31]	1,440
ReLU6-22	[1, 144, 31, 31]	0
Conv2d-23	[1, 32, 31, 31]	4,640
InvertedResidual-24	[1, 32, 31, 31]	0
Conv2d-25	[1, 192, 31, 31]	6,336
ReLU6-26	[1, 192, 31, 31]	0
Conv2d-27	[1, 192, 31, 31]	1,920
ReLU6-28	[1, 192, 31, 31]	0
Conv2d-29	[1, 32, 31, 31]	6,176
InvertedResidual-30	[1, 32, 31, 31]	0
Conv2d-31	[1, 192, 31, 31]	6,336
ReLU6-32	[1, 192, 31, 31]	0
Conv2d-33	[1, 192, 31, 31]	1,920
ReLU6-34	[1, 192, 31, 31]	0
Conv2d-35	[1, 32, 31, 31]	6,176
InvertedResidual-36	[1, 32, 31, 31]	0
Conv2d-37	[1, 192, 31, 31]	6,336
ReLU6-38	[1, 192, 31, 31]	0
Conv2d-39	[1, 192, 16, 16]	1,920
ReLU6-40	[1, 192, 16, 16]	0
Conv2d-41	[1, 64, 16, 16]	12,352
InvertedResidual-42	[1, 64, 16, 16]	0
Conv2d-43	[1, 384, 16, 16]	24,960
ReLU6-44	[1, 384, 16, 16]	0
Conv2d-45	[1, 384, 16, 16]	3,840
ReLU6-46	[1, 384, 16, 16]	0
Conv2d-47	[1, 64, 16, 16]	24,640
InvertedResidual-48	[1, 64, 16, 16]	0
Conv2d-49	[1, 384, 16, 16]	24,960
ReLU6-50	[1, 384, 16, 16]	0
Conv2d-51	[1, 384, 16, 16]	3,840
ReLU6-52	[1, 384, 16, 16]	0

PyTorch modeli 0 hatayla oluştu

ONNX’e export edildi
(mobilenet_v2_my.onnx oluşturuldu)

“mobilenet_v2.onnx” referans
dosyası ile export ettiğimiz
model arasında birebir numerik
eşleşme var

Conv2d-53	[1, 64, 16, 16]	24,640
InvertedResidual-54	[1, 64, 16, 16]	0
Conv2d-55	[1, 384, 16, 16]	24,960
ReLU6-56	[1, 384, 16, 16]	0
Conv2d-57	[1, 384, 16, 16]	3,840
ReLU6-58	[1, 384, 16, 16]	0
Conv2d-59	[1, 64, 16, 16]	24,640
InvertedResidual-60	[1, 64, 16, 16]	0
Conv2d-61	[1, 384, 16, 16]	24,960
ReLU6-62	[1, 384, 16, 16]	0
Conv2d-63	[1, 384, 16, 16]	3,840
ReLU6-64	[1, 384, 16, 16]	0
Conv2d-65	[1, 96, 16, 16]	36,960
InvertedResidual-66	[1, 96, 16, 16]	0
Conv2d-67	[1, 576, 16, 16]	55,872
ReLU6-68	[1, 576, 16, 16]	0
Conv2d-69	[1, 576, 16, 16]	5,760
ReLU6-70	[1, 576, 16, 16]	0
Conv2d-71	[1, 96, 16, 16]	55,392
InvertedResidual-72	[1, 96, 16, 16]	0
Conv2d-73	[1, 576, 16, 16]	55,872
ReLU6-74	[1, 576, 16, 16]	0
Conv2d-75	[1, 576, 16, 16]	5,760
ReLU6-76	[1, 576, 16, 16]	0
Conv2d-77	[1, 96, 16, 16]	55,392
InvertedResidual-78	[1, 96, 16, 16]	0
Conv2d-79	[1, 576, 16, 16]	55,872
ReLU6-80	[1, 576, 16, 16]	0
Conv2d-81	[1, 576, 8, 8]	5,760
ReLU6-82	[1, 576, 8, 8]	0
Conv2d-83	[1, 160, 8, 8]	92,320
InvertedResidual-84	[1, 160, 8, 8]	0
Conv2d-85	[1, 960, 8, 8]	154,560
ReLU6-86	[1, 960, 8, 8]	0
Conv2d-87	[1, 960, 8, 8]	9,600
ReLU6-88	[1, 960, 8, 8]	0
Conv2d-89	[1, 160, 8, 8]	153,760
InvertedResidual-90	[1, 160, 8, 8]	0
Conv2d-91	[1, 960, 8, 8]	154,560
ReLU6-92	[1, 960, 8, 8]	0
Conv2d-93	[1, 960, 8, 8]	9,600
ReLU6-94	[1, 960, 8, 8]	0
Conv2d-95	[1, 160, 8, 8]	153,760
InvertedResidual-96	[1, 160, 8, 8]	0
Conv2d-97	[1, 960, 8, 8]	154,560
ReLU6-98	[1, 960, 8, 8]	0
Conv2d-99	[1, 960, 8, 8]	9,600
ReLU6-100	[1, 960, 8, 8]	0
Conv2d-101	[1, 320, 8, 8]	307,520
InvertedResidual-102	[1, 320, 8, 8]	0
Conv2d-103	[1, 1280, 8, 8]	410,880
ReLU6-104	[1, 1280, 8, 8]	0
Linear-105	[1, 1000]	1,281,000
=====		
Total params: 3,487,816		

Bu, modelin toplam öğrenilebilir parametre sayısıdır.
Bu sayı orijinal MobileNetV2 ile birebir aynı $\rightarrow \approx 3.48\text{M}$ parametre.

```
=====
Total params: 3,487,816
Trainable params: 3,487,816
Non-trainable params: 0
-----
Input size (MB): 0.68
Forward/backward pass size (MB): 124.62
Params size (MB): 13.30
Estimated Total Size (MB): 138.61
-----
```

Bu, bir forward + backward pass sırasında bellekte tutulan tüm geçici ara tensörlerin toplam boyutu.

Conv2d

PyTorch nn.Conv2d

nn.Conv2d(input, output, 3,
stride, 1, bias=True),

group = 1 → Normal Convolution
Tüm giriş kanalları, tüm filtrelerle
bağlantılıdır.

input_channels = 32

out_channels = 16

group = 1

Demek ki:

Her bir filtre 32 kanalı da görür.

```
class Conv2d : public Layer {
// Constructor with default parameters for a convolution layer
public:
    explicit Conv2d(int in_channels=1,
                    int out_channels=1,
                    int kernel_size=1,
                    int stride=1,
                    int padding=0,
                    int group=1) :
        // Member initializer list
        in_channels(in_channels / group),
        out_channels(out_channels),
        kernel_size(kernel_size),
        stride(stride),
        padding(padding),
        group(group) {

        // Total number of weights in kernel tensor
        size = out_channels * in_channels * kernel_size * kernel_size;
        // Allocate memory for weights and bias
        weight = new float[size];
        bias = new float[out_channels];
    }
}
```

group = in_channels (ya da hidden_dim) → Depthwise Convolution
Bu depthwise conv'dur.

Örnek:

- input_channels = 32
- out_channels = 32
- group = 32

Demek ki:

Her giriş kanalı yalnızca kendi filtresi ile konvolüsyon yapar.
Kanallar birbirine karışmaz.

```
nn.Conv2d(hidden_dim, hidden_dim, 3, stride, 1, groups=hidden_dim)
```

```

// Copy assignment operator. Takes parameter by value (copy)
Conv2d &operator=(Conv2d conv2d) {
    // Copy metadata from the given Conv2d
    in_channels = conv2d.getInChannels();
    out_channels = conv2d.getOutChannels();
    kernel_size = conv2d.getKernelSize();
    stride = conv2d.getStride();
    padding = conv2d.getPadding();
    group = conv2d.getGroup();

    // Delete previous arrays to avoid memory leaks
    delete []weight;
    delete []bias;

    // Allocate new array with updated size
    size = out_channels * in_channels * kernel_size * kernel_size;
    weight = new float[size];
    bias = new float[out_channels];

    // Returns modified object
    return *this;
}

```

Conv2d A, B;
A = B;

Normalde C++ default bir kopyalama operatörü yaratır ama dinamik bellek var diye manuel yazmamız gerekiyor.

Layer metadata'sını kopyalıyor

Eski belleği siliyor → memory leak yok

Yeni boyuta uygun weight/bias memory allocate ediyor

Yani amaç sadece aynı layer'ın boş bir kopyasını üretmek, ağırlıkları taşımak değil.

```

// Destructor frees dynamically allocated memory
~Conv2d() {
    delete []weight;
    delete []bias;
}

// Prints Conv2d parameters with colored formatting.
void show() {
    std::cout << "\033[36mConv2d(\033[0m"
    << in_channels << ", "
    << out_channels << ", "
    << kernel_size << ", "
    << stride << ", "
    << padding << ", "
    << group
    << "\033[36m)\033[0m";
}

// Returns type code for Conv2d
int getType() {
    return 0;
}

```

C++'ta dinamik olarak ayrılan belleği manuel olarak serbest bırakıyoruz

Conv2d(3, 32, 3, 1, 1, 1)

Parametre	Anlamı
in_channels	giriş kanal sayısı (ör: 3 → RGB)
out_channels	çıkış kanal sayısı
kernel_size	filtre boyutu (3×3)
stride	kaydırma miktarı
padding	kenarlara eklenen dolgular
group	group convolution parametresi

```
// Returns a vector of parameters with their shapes
std::vector<std::pair<float*, std::vector<int>>> getParameters() {
    return {
        std::make_pair(weight, std::vector<int>{out_channels, in_channels, kernel_size, kernel_size}),
        std::make_pair(bias, std::vector<int>{out_channels})
    };
}

// Getter methods
float *getWeight() { return weight; }
float *getBias() { return bias; }
int getInChannels() { return in_channels; }
int getOutChannels() { return out_channels; }
int getKernelSize() { return kernel_size; }
int getStride() { return stride; }
int getPadding() { return padding; }
int getGroup() { return group; }
int getSize() { return size; }

// Private members, internal state of Conv2d layer
private:
    float *weight;
    float *bias;
    int in_channels;
    int out_channels;
    int kernel_size;
    int stride;
    int padding;
    int group;
    int size;
};

#endif // CONV_2D
```

Parametre	Bellek Adresi	Shape
Weight	float* weight	[out_channels, in_channels, k, k]
Bias	float* bias	[out_channels]

```

1 // Header guard to prevent multiple inclusion of the same file.
2 #ifndef GLOBAL_AVERAGE_POOL
3 #define GLOBAL_AVERAGE_POOL
4
5 // Include the base Layer class.
6 #include "Layer.h"
7
8 // Defines a class named GlobalAveragePool, derived from Layer.
9 class GlobalAveragePool : public Layer {
10 public:
11     // Default constructor with no internal initialization needed.
12     explicit GlobalAveragePool() {}
13     // Destructor does nothing since no dynamic memory is allocated.
14     ~GlobalAveragePool() {}
15     // Prints the name "GlobalAveragePool" in blue using ANSI escape codes.
16     void show() {
17         std::cout << "\033[34mGlobalAveragePool\033[0m";
18     }
19     // Returns the identifier code 1 for this layer type.
20     int getType() {
21         return 1;
22     }
23 };
24
25 #endif // GLOBAL_AVERAGE_POOL_

```

[B, C, H, W] → [B, C]

placeholder şu an için

```

1 // Header guard to prevent multiple inclusion of this file
2 #ifndef RESIDUAL_ADD
3 #define RESIDUAL_ADD
4
5 // Includes base layer class definition
6 #include "Layer.h"
7
8 // Defines a class which inherits from layer
9 class ResidualAdd : public Layer {
10 public:
11     // Constructor initializes residual pointer to nullptr
12     explicit ResidualAdd() { residual = nullptr; }
13     // Destructor simply resets pointer to nullptr
14     ~ResidualAdd() { residual = nullptr; }
15     // Returns a pointer to the residual layer
16     Layer *getResidual() { return residual; }
17     // Sets the internal residual pointer, linking another layer into the skip connection.
18     void setResidual(Layer &resLayer) { residual = &resLayer; }
19     // Prints the ResidualAdd operator, if a residual layer exists it calls its show function too.
20     void show() {
21         std::cout << "\033[33mResidualAdd(\033[0m";
22         if (residual) {
23             residual->show();
24         }
25         std::cout << "\033[33m)\033[0m";
26     }
27     // Returns type code 4
28     int getType() {
29         return 4;
30     }
31     // Stores a pointer to another layer that will be added element wise to the main input
32 private:
33     Layer *residual;
34 };
35
36 #endif // RESIDUAL_ADD

```

placeholder şu an için

```

1  // Header guard to prevent this header file from being included multiple times
2  #ifndef RELU6
3  #define RELU6
4
5  // Includes the base Layer class this activation function inherits from.
6  #include "Layer.h"
7
8  // Defines the ReLU6 class, inheriting from the abstract Layer class.
9  class ReLU6 : public Layer {
10 public:
11     // Default constructor, no special initialization needed.
12     explicit ReLU6() {}
13     // Destructor does nothing since this class allocates no dynamic memory.
14     ~ReLU6() {}
15     // Prints the name "ReLU6" in purple
16     void show() {
17         std::cout << "\033[35mReLU6\033[0m";
18     }
19     // Returns an integer ID (3) representing the type of this layer.
20     int getType() {
21         return 3;
22     }
23 };
24
25 #endif // RELU6

```

placeholder şu an için

```
// Header guard to avoid including this file multiple times.
#ifndef LINEAR
#define LINEAR

// Includes the base class Layer.
#include "Layer.h"

// Defines a Linear layer class that inherits from Layer.
class Linear : public Layer {
public:
    // Constructor with default parameters. Initializes input and output size.
    explicit Linear(int in_features=1, int out_features=1) :
        in_features(in_features), out_features(out_features) {

        // Total number of weights = rows x columns = out_features x in_features.
        size = out_features * in_features;
        // Allocates dynamic memory for weight matrix and bias vector.
        weight = new float[size];
        bias = new float[out_features];
    }

    // Copy assignment using pass-by-value (copy-swap idiom style).
    Linear &operator=(Linear linear) {
        // Copies metadata (layer dimensions).
        in_features = linear.getInFeatures();
        out_features = linear.getOutFeatures();

        // Deletes previous arrays to avoid memory leaks.
        delete []weight;
        delete []bias;

        // Allocates new memory according to the new layer dimensions.
        size = out_features * in_features;
        weight = new float[size];
        bias = new float[out_features];
        // Returns reference to the updated object.
        return *this;
    }
};
```

nn.Linear katmanının C++ karşılığıdır

Parametre	Anlamı
in_features	Giriş boyutu (ör: 1280)
out_features	Çıkış boyutu (ör: 1000 – ImageNet)

Linear katmanı = Matris çarpımı
W boyutu → [out_features, in_features]

Bu operator ağırlıkları kopyalamaz, sadece bellek ve metadata kopyalanır. Ağırlıkları yüklemek MobileNetV2'nin kendi yükleme fonksiyonunda yapılır.

```

~Linear() {
    // Releases dynamically allocated memory.
    delete []weight;
    delete []bias;
}

// Prints the Linear layer information in green.
void show() {
    std::cout << "\033[32mLinear(\033[0m"
               << in_features << ", "
               << out_features
               << "\033[32m)\033[0m";
}

// Returns ID code 2 for a Linear layer.
int getType() {
    return 2;
}

// Returns parameters along with their shapes.
std::vector<std::pair<float*, std::vector<int>>> getParameters() {
    return {
        std::make_pair(weight, std::vector<int>{out_features, in_features}),
        std::make_pair(bias, std::vector<int>{out_features})
    };
}

// Simple getter functions to access internal data.
float *getWeight() { return weight; }
float *getBias() { return bias; }
int getInFeatures() { return in_features; }
int getOutFeatures() { return out_features; }
int getSize() { return size; }

// Internal storage of the Linear layer
private:
    float *weight;
    float *bias;
    int in_features;
    int out_features;
    int size;
};

```

C++'ta new ile açtığımız her şeyi delete ile silmeliyiz yoksa bellek sızıntısı olur.

Linear layer'ın weight ve bias değerlerinin pointer'ını
Aynı zamanda tensör şekillerini
döndürüyor

ONNX'den gelen ağırlıkları doğru
katmana doğru boyutta yazabiliriz

```

1 // Header guard preventing the file from being included multiple times.
2 #ifndef LAYER
3 #define LAYER
4
5 #include <memory>
6 #include <vector>
7 #include <cassert>
8 #include <iostream>
9
10 // Defines an abstract base class called Layer.
11 class Layer {
12 public:
13     // Initializes a layer with: no next layer yet, this layer is not a bypass connection by defaults
14     Layer() {
15         next = nullptr;
16         bypass = false;
17     }
18     // Destructor simply nulls the pointer.
19     ~Layer() { next = nullptr; }
20
21     // Returns the next layer in the chain.
22     Layer *getNext() { return next; }
23
24     // Sets the next layer and returns it.
25     Layer *setNext(Layer &nextLayer) {
26         next = &nextLayer;
27         return next;
28     }
29
30     // Returns whether this layer is part of a residual skip connection.
31     bool getIsBypass() { return bypass; }
32
33     // Marks this layer as part of a bypass path.
34     void setIsBypass() { bypass = true; }
35
36     // Base class version is not implemented. Derived class must override this.
37     Layer *getResidual() {
38         throw std::runtime_error("not implement error\n");
39     }
40
41     // Derived classes implement residual logic.
42     void setResidual(Layer &resLayer) {
43         throw std::runtime_error("not implement error\n");
44     }

```

Her layer bir sonraki layer'ı gösterir.
Çünkü PyTorch'ta model bir grafiktir, ama burada biz elle zincir yapıyoruz:

Conv → ReLU → Conv → Add →
AvgPool → Linear

bypass → skip connection işareti.

Residual bloklarda:

$$y = x + F(x)$$

Buradaki $x \rightarrow$ bypass path.

```
// Every layer must implement a show() function to print layer details.
virtual void show() {
    throw std::runtime_error("not implement error\n");
}

// Each derived layer returns a unique ID.
virtual int getType() {
    throw std::runtime_error("not implement error\n");
}

// Returns layer parameters (weights, biases) and their shapes.
virtual std::vector<std::pair<float*, std::vector<int>>>> getParameters() {
    throw std::runtime_error("not implement error\n");
};
virtual float *getWeight() {
    throw std::runtime_error("not implement error\n");
}
virtual float *getBias() {
    throw std::runtime_error("not implement error\n");
}
virtual int getInChannels() {
    throw std::runtime_error("not implement error\n");
}
virtual int getOutChannels() {
    throw std::runtime_error("not implement error\n");
}
virtual int getKernelSize() {
    throw std::runtime_error("not implement error\n");
}
virtual int getStride() {
    throw std::runtime_error("not implement error\n");
}
virtual int getPadding() {
    throw std::runtime_error("not implement error\n");
}
virtual int getGroup() {
    throw std::runtime_error("not implement error\n");
}
virtual int getSize() {
    throw std::runtime_error("not implement error\n");
}
virtual int getInFeatures() {
    throw std::runtime_error("not implement error\n");
}
```

Bu bir abstract class yani diğer
conv2d gibi layerlar bunu base
alıp bundaki fonksiyonları
dolduruyorlar eğer
doldurmazsalar hata alıyoruz
buradaki fonksiyonları

```

1 // Header guard.
2 #ifndef MOBILE_NET_V2
3 #define MOBILE_NET_V2
4
5 // Includes all layer types used in MobileNetV2, plus standard C++ headers.
6 #include "Conv2d.h"
7 #include "GlobalAveragePool.h"
8 #include "Linear.h"
9 #include "ReLU6.h"
10 #include "ResidualAdd.h"
11 #include <fstream>
12 #include <cassert>
13 #include <string>
14
15 // Defines fixed counts of Conv2d, ReLU6, and ResidualAdd layers in MobileNetV2.
16 // Used to allocate fixed size arrays.
17 #define NUM_CONV 52
18 #define NUM_RELU 35
19 #define NUM_ADD 10
20
21 // Defines the maximum buffer needed to store intermediate feature maps.
22 // Allows pre allocating workspace buffers during inference.
23 // For batch_size = 1, [1, 96, 122, 122] -> 1 * 96 * 122 * 122 = 1428864
24 #define MAX_SIZE_INTERMEDIATE 1428864
25

```

MobileNetV2'nin
PyTorch/ONNX
versiyonunda:

toplam 52 tane Conv2d

toplam 35 tane ReLU6

toplam 10 tane ResidualAdd

(skip connection)

olduğu biliniyor.

değeri, MobileNetV2'nin bütün
forward pass'i boyunca ortaya
çıkan en büyük feature map'ın
boyutudur; bu da girişten sonra
oluşan $1 \times 96 \times 122 \times 122$
tensöründen gelir.

```
// Start of MobileNetV2 neural network class.
```

```
class MobileNetV2 {
```

```
public:
```

```
    // Build model topology and load parameters
```

```
    explicit MobileNetV2(std::string weights_path = "./weights/");
```

```
    // Get entry
```

```
    Layer *getEntry() { return entry; }
```

```
    // Get layers
```

```
    Conv2d *getConv2dLayers() { return Conv2dLayers; }
```

```
    ReLU6 *getReLU6Layers() { return ReLU6Layers; }
```

```
    ResidualAdd *getResidualAddLayers() { return ResidualAddLayers; }
```

```
    GlobalAveragePool getGlobalAveragePoolLayer() { return GlobalAveragePoolLayer; }
```

```
    Linear getLinearLayer() { return LinearLayer; }
```

```
    // Show
```

```
    void show();
```

```
    // Get maximum size of intermediate representations
```

```
    int getMaxIntermediateSize() { return MAX_SIZE_INTERMEDIATE; }
```

bütün layer'lar bağlantı kurulur

ağırlık dosyaları yüklenir

inverted residual bloklar oluşturulur

Modelin ilk katmanı döner.

```

int getMaxIntermediateSize() { return MAX_SIZE_INTERMEDIATE; }

private:
    // Path where weight files are stored.
    std::string weights_path;
    // first layer, used while building layer sequence (like a cursor)
    Layer *entry;
    Layer *curr;
    // Arrays of layers – MobileNetV2 has a fixed number of each layer type.
    Conv2d Conv2dLayers [NUM_CONV];
    ReLU6 ReLU6Layers [NUM_RELU];
    ResidualAdd ResidualAddLayers [NUM_ADD];
    // The final two layers of MobileNetV2
    GlobalAveragePool GlobalAveragePoolLayer;
    Linear LinearLayer;
    // Loads convolution weights/bias from file #idx.
    void GetConvWeights(float *weight, float *bias, int idx);
    // Loads fully connected layer weights (for the final classifier).
    void GetGemmWeight(float *weight, float *bias);
    // This function builds one inverted residual block from MobileNetV2:
    void BuildInvertedResidual(int input_dim, int output_dim, int expansion_ratio, int stride,
                               int &i_conv, int &i_relu, int &i_add, int i);
};

#endif // MOBILE_NET_V2

```

modelin iç yapısı, dışarıdan erişilemez

Ağırlık path'i

Layer pointer'ları

Layer array'leri

Ağırlık yükleyen
fonksiyonlar

1x1 conv (expand)

depthwise conv

1x1 conv (project)

varsa skip connection

Bu fonksiyon bir adet blok inşa eder.

```

1  #include "MobileNetCuda.h"
2  #include <iomanip>
3
4  // Initialize MobileNetV2 with a path to pretrained weights.
5  MobileNetV2::MobileNetV2(std::string weights_path) : weights_path(weights_path) {
6
7      int i_conv = 0, i_relu = 0, i_add = 0;
8
9      // First Conv2d + ReLU6
10     // First convolution:
11     // Input channels = 3 (RGB image)
12     // Output = 32
13     // Kernel = 3x3
14     // Stride = 2 → downsampling
15     // Padding = 1
16     Conv2dLayers[i_conv] = Conv2d(3, 32, 3, 2, 1);
17     // Load pretrained conv weights from file.
18     GetConvWeights(Conv2dLayers[i_conv].getWeight(), Conv2dLayers[i_conv].getBias(), i_conv);
19     // curr = currently last inserted layer, entry = first layer in the model, i_conv++ = move to next conv index
20     curr = entry = &Conv2dLayers[i_conv++];
21     // Link the ReLU6 activation after the first conv.
22     curr = curr->setNext(ReLU6Layers[i_relu++]);
23

```

Katman sayaçları

```
// Inverted Residuals
```

```
std::vector<std::vector<int>> inverted_residual_settings{
```

```
    // t, c, n, s
```

```
    // t: expansion ratio
```

```
    // c: number of output channels
```

```
    // n: repeated times of identical layers
```

```
    // s: stride of first layer
```

```
    {1, 16, 1, 1},
```

```
    {6, 24, 2, 2},
```

```
    {6, 32, 3, 2},
```

```
    {6, 64, 4, 2},
```

```
    {6, 96, 3, 1},
```

```
    {6, 160, 3, 2},
```

```
    {6, 320, 1, 1},
```

```
};
```

```
class MobileNetV2(nn.Module):
```

```
    def __init__(self, num_class=1000, onnx_path='mobilenet_v2.onnx'):
```

```
        super(MobileNetV2, self).__init__()
```

```
        input_channel = 32
```

```
        last_channel = 1280
```

```
        inverted_residual_settings = [
```

```
            # t, c, n, s
```

```
            # t: expansion ratio
```

```
            # c: number of output channels
```

```
            # n: repeated times of identical layers
```

```
            # s: stride of first layer
```

```
            [1, 16, 1, 1],
```

```
            [6, 24, 2, 2],
```

```
            [6, 32, 3, 2],
```

```
            [6, 64, 4, 2],
```

```
            [6, 96, 3, 1],
```

```
            [6, 160, 3, 2],
```

```
            [6, 320, 1, 1],
```

```
        ]
```

```

// Track current number of channels
int input_dim = 32;
int stride;

// For each block group: first block uses given stride and repeated blocks use stride 1
for (auto &setting : inverted_residual_settings) {
    for (int i = 0; i < setting[2]; i++) {
        stride = (i == 0) ? setting[3] : 1;
        // Build one inverted residual block and connect it into the layer chain
        BuildInvertedResidual(input_dim, setting[1], setting[0], stride,
                               i_conv, i_relu, i_add, i);
        // Update input dimension for next block group
        input_dim = setting[1];
    }
}

// Last Conv2d + ReLU6
Conv2dLayers[i_conv] = Conv2d(320, 1280, 1, 1, 0);
GetConvWeights(Conv2dLayers[i_conv].getWeight(), Conv2dLayers[i_conv].getBias(), i_conv);
curr = curr->setNext(Conv2dLayers[i_conv++]);
curr = curr->setNext(ReLU6Layers[i_relu++]);

// Global Average Pool
curr = curr->setNext(GlobalAveragePoolLayer);

// Linear classifier 1280 feautre, 1000 class
LinearLayer = Linear(1280, 1000);
GetGemmWeight(LinearLayer.getWeight(), LinearLayer.getBias());
curr = curr->setNext(LinearLayer);

// Assert index
assert(i_conv == NUM_CONV);
assert(i_relu == NUM_RELU);
assert(i_add == NUM_ADD);

```

Tüm inverted residual
bloklarını kurma

Son Konvolüsyon + ReLU6

Global Average Pool ekleniyor

Linear Katman (Classifier)

```

// Traverse the linked list of layers and print them.
void MobileNetV2::show() {
    curr = entry;
    int i = 0;
    while (curr) {
        std::cout << std::setw(2) << i++;
        std::cout << ": ";
        curr->show();
        std::cout << std::endl;
        curr = curr->getNext();
    }
}

// Load weights from files like: 0.w, 0.b
void MobileNetV2::GetConvWeights(float *weight, float *bias, int idx) {
    std::ifstream infile;
    int k;

    // weight
    infile.open(weights_path + "conv/" + std::to_string(idx) + ".w");
    assert(infile.is_open());
    k = 0;
    while (!infile.eof()) {
        infile >> weight[k++];
    }
    infile.close();

    // bias
    infile.open(weights_path + "conv/" + std::to_string(idx) + ".b");
    assert(infile.is_open());
    k = 0;
    while (!infile.eof()) {
        infile >> bias[k++];
    }
    infile.close();
}

```

Modeldeki tüm layer'ları sırayla ekrana yazıyor

entry'den başlar (modelin ilk layer'ı:
ilk Conv2d)

Her seferinde:

- layer'ın index'ini yazar
- o layer'ın tipi ne ise onu yazdırır (Conv2d, ReLU6, GlobalAvgPool, vs.)
- curr = curr->getNext() ile bir sonraki layer'a geçer
- Son layer'a kadar tek tek yazdırır.

```

// Traverse the linked list of layers and print them.
void MobileNetV2::show() {
    curr = entry;
    int i = 0;
    while (curr) {
        std::cout << std::setw(2) << i++;
        std::cout << ": ";
        curr->show();
        std::cout << std::endl;
        curr = curr->getNext();
    }
}

// Load weights from files like: 0.w, 0.b
void MobileNetV2::GetConvWeights(float *weight, float *bias, int idx) {
    std::ifstream infile;
    int k;

    // weight
    infile.open(weights_path + "conv/" + std::to_string(idx) + ".w");
    assert(infile.is_open());
    k = 0;
    while (!infile.eof()) {
        infile >> weight[k++];
    }
    infile.close();

    // bias
    infile.open(weights_path + "conv/" + std::to_string(idx) + ".b");
    assert(infile.is_open());
    k = 0;
    while (!infile.eof()) {
        infile >> bias[k++];
    }
    infile.close();
}

```

ONNX'tan dışa aktarılan .w →
weight dosyaları, .b → bias
dosyalarını okumak

0.w → 0. convolution layer'ın weight değerleri
0.b → bias

1.w → 1. conv layer'ın weight'leri

Her Conv layer'ın weight ve bias'ını
dosyadan okur

Bunları C++ içindeki layer'ın float*
weight ve float* bias belleğine yazar

ONNX → C++ modele tam doğru weight
eşleştirmesi sağlar

```

// Load linear layer weights
void MobileNetV2::GetGemmWeight(float *weight, float *bias) {
    std::ifstream infile;
    int k;

    // weight
    infile.open(weights_path + "gemm/0.w");
    assert(infile.is_open());
    k = 0;
    while (!infile.eof()) {
        infile >> weight[k++];
    }
    infile.close();

    // bias
    infile.open(weights_path + "gemm/0.b");
    assert(infile.is_open());
    k = 0;
    while (!infile.eof()) {
        infile >> bias[k++];
    }
    infile.close();
}

```

Bu fonksiyon,
MobileNetV2'nin son
Linear (Fully Connected)
katmanının ağırlıklarını ve
bias değerlerini .w ve .b
dosyalarından okuyup
belleğe yükler.

GEMM (General Matrix Multiply)

$$y=Wx+b$$

```
// The core block function
```

```
void MobileNetV2::BuildInvertedResidual(int input_dim, int output_dim, int expansion_ratio, int stride,  
int &i_conv, int &i_relu, int &i_add, int i) {
```

```
int hidden_dim = input_dim * expansion_ratio;
```

Genişletilmiş kanal sayısını hesapla

```
// Depthwise convolution - ReLU6 - 1x1 projection convolution
```

```
if (expansion_ratio == 1) {
```

Bu durumda blok genişleme yapmaz:

```
// depthwise convolution
```

```
Conv2dLayers[i_conv] = Conv2d(hidden_dim, hidden_dim, 3, stride, 1, hidden_dim);
```

```
GetConvWeights(Conv2dLayers[i_conv].getWeight(), Conv2dLayers[i_conv].getBias(), i_conv);
```

```
curr = curr->setNext(Conv2dLayers[i_conv++]);
```

```
curr = curr->setNext(ReLU6Layers[i_relu++]);
```

```
// pointwise convolution, linear
```

```
Conv2dLayers[i_conv] = Conv2d(hidden_dim, output_dim, 1, 1, 0);
```

```
GetConvWeights(Conv2dLayers[i_conv].getWeight(), Conv2dLayers[i_conv].getBias(), i_conv);
```

```
curr = curr->setNext(Conv2dLayers[i_conv++]);
```

```
} else { // Pointwise expansion conv - Depthwise - Pointwise projection
```

```
// pointwise convolution, ReLU6
```

```
Conv2dLayers[i_conv] = Conv2d(input_dim, hidden_dim, 1, 1, 0);
```

```
GetConvWeights(Conv2dLayers[i_conv].getWeight(), Conv2dLayers[i_conv].getBias(), i_conv);
```

```
curr = curr->setNext(Conv2dLayers[i_conv++]);
```

```
curr = curr->setNext(ReLU6Layers[i_relu++]);
```

```
// depthwise convolution
```

```
Conv2dLayers[i_conv] = Conv2d(hidden_dim, hidden_dim, 3, stride, 1, hidden_dim);
```

```
GetConvWeights(Conv2dLayers[i_conv].getWeight(), Conv2dLayers[i_conv].getBias(), i_conv);
```

```
curr = curr->setNext(Conv2dLayers[i_conv++]);
```

```
curr = curr->setNext(ReLU6Layers[i_relu++]);
```

```
// pointwise convolution, linear
```

```
Conv2dLayers[i_conv] = Conv2d(hidden_dim, output_dim, 1, 1, 0);
```

```
GetConvWeights(Conv2dLayers[i_conv].getWeight(), Conv2dLayers[i_conv].getBias(), i_conv);
```

```
curr = curr->setNext(Conv2dLayers[i_conv++]);
```

```
}
```

Eğer expansion_ratio == 1

3×3 depthwise conv +

ReLU6

1×1 conv

Skip connection (varsa)

Eğer expansion_ratio > 1

1×1 conv (expand)

+

ReLU6

+

3×3 depthwise conv

+

ReLU6

+

1×1 conv (projection)

+

Skip connection (varsa)

```
// Residual only possible when stride = 1 and input and output channels match
if (stride == 1 && input_dim == output_dim) {
    if (i == 1) { // Correctly links skip connection depending on whether we are inside a repeated block
        ResidualAddLayers[i_add].setResidual(Conv2dLayers[i_conv - 4]);
        Conv2dLayers[i_conv - 4].setIsBypass();
    } else {
        ResidualAddLayers[i_add].setResidual(ResidualAddLayers[i_add - 1]);
        ResidualAddLayers[i_add - 1].setIsBypass();
    }
    // Insert ResidualAdd layer into the network
    curr = curr->setNext(ResidualAddLayers[i_add++]);
}
}
```

Eğer residual bağlantı mümkünse ekler

```
bash-5.1$ ./check.out  
bash-5.1$ ./show.out
```

- Diskten weight dosyalarını (conv/*.w, conv/*.b) okuyor.
- MobileNetV2'deki memory'deki weight ve bias ile karşılaştırıyor.
- Eğer hiçbir hata yoksa hiçbir şey yazmıyor → sessiz geçiyor.
- Bir hata olursa program crash ediyor ve error geliyor.

Input	Operator	t	Output Channels	Repeat	Stride
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1×1	-	1280	1	1
$7^2 \times 1280$	avgpool 7×7	-	-	1	-

```

bash-5.1$ ./show.out
0: Conv2d(3, 32, 3, 2, 1, 1)
1: ReLU6
2: Conv2d(1, 32, 3, 1, 1, 32)
3: ReLU6
4: Conv2d(32, 16, 1, 1, 0, 1)
5: Conv2d(16, 96, 1, 1, 0, 1)
6: ReLU6
7: Conv2d(1, 96, 3, 2, 1, 96)
8: ReLU6
9: Conv2d(96, 24, 1, 1, 0, 1)
10: Conv2d(24, 144, 1, 1, 0, 1)
11: ReLU6
12: Conv2d(1, 144, 3, 1, 1, 144)
13: ReLU6
14: Conv2d(144, 24, 1, 1, 0, 1)
15: ResidualAdd(Conv2d(96, 24, 1, 1, 0, 1))
16: Conv2d(24, 144, 1, 1, 0, 1)
17: ReLU6
18: Conv2d(1, 144, 3, 2, 1, 144)
19: ReLU6
20: Conv2d(144, 32, 1, 1, 0, 1)
21: Conv2d(32, 192, 1, 1, 0, 1)
22: ReLU6
23: Conv2d(1, 192, 3, 1, 1, 192)
24: ReLU6
25: Conv2d(192, 32, 1, 1, 0, 1)
26: ResidualAdd(Conv2d(144, 32, 1, 1, 0, 1))
27: Conv2d(32, 192, 1, 1, 0, 1)
28: ReLU6
29: Conv2d(1, 192, 3, 1, 1, 192)
30: ReLU6
31: Conv2d(192, 32, 1, 1, 0, 1)
32: ResidualAdd(ResidualAdd(Conv2d(144, 32, 1, 1, 0, 1)))
33: Conv2d(32, 192, 1, 1, 0, 1)
34: ReLU6
35: Conv2d(1, 192, 3, 2, 1, 192)
36: ReLU6
37: Conv2d(192, 64, 1, 1, 0, 1)
38: Conv2d(64, 384, 1, 1, 0, 1)
39: ReLU6
40: Conv2d(1, 384, 3, 1, 1, 384)
41: ReLU6
42: Conv2d(384, 64, 1, 1, 0, 1)
43: ResidualAdd(Conv2d(192, 64, 1, 1, 0, 1))
44: Conv2d(64, 384, 1, 1, 0, 1)
45: ReLU6
46: Conv2d(1, 384, 3, 1, 1, 384)
47: ReLU6
48: Conv2d(384, 64, 1, 1, 0, 1)
49: ResidualAdd(ResidualAdd(Conv2d(192, 64, 1, 1, 0, 1)))
50: Conv2d(64, 384, 1, 1, 0, 1)
51: ReLU6
52: Conv2d(1, 384, 3, 1, 1, 384)
53: ReLU6
54: Conv2d(384, 64, 1, 1, 0, 1)
55: ResidualAdd(ResidualAdd(ResidualAdd(Conv2d(192, 64, 1, 1, 0, 1))))

```

```

56: Conv2d(64, 384, 1, 1, 0, 1)
57: ReLU6
58: Conv2d(1, 384, 3, 1, 1, 384)
59: ReLU6
60: Conv2d(384, 96, 1, 1, 0, 1)
61: Conv2d(96, 576, 1, 1, 0, 1)
62: ReLU6
63: Conv2d(1, 576, 3, 1, 1, 576)
64: ReLU6
65: Conv2d(576, 96, 1, 1, 0, 1)
66: ResidualAdd(Conv2d(384, 96, 1, 1, 0, 1))
67: Conv2d(96, 576, 1, 1, 0, 1)
68: ReLU6
69: Conv2d(1, 576, 3, 1, 1, 576)
70: ReLU6
71: Conv2d(576, 96, 1, 1, 0, 1)
72: ResidualAdd(ResidualAdd(Conv2d(384, 96, 1, 1, 0, 1)))
73: Conv2d(96, 576, 1, 1, 0, 1)
74: ReLU6
75: Conv2d(1, 576, 3, 2, 1, 576)
76: ReLU6
77: Conv2d(576, 160, 1, 1, 0, 1)
78: Conv2d(160, 960, 1, 1, 0, 1)
79: ReLU6
80: Conv2d(1, 960, 3, 1, 1, 960)
81: ReLU6
82: Conv2d(960, 160, 1, 1, 0, 1)
83: ResidualAdd(Conv2d(576, 160, 1, 1, 0, 1))
84: Conv2d(160, 960, 1, 1, 0, 1)
85: ReLU6
86: Conv2d(1, 960, 3, 1, 1, 960)
87: ReLU6
88: Conv2d(960, 160, 1, 1, 0, 1)
89: ResidualAdd(ResidualAdd(Conv2d(576, 160, 1, 1, 0, 1)))
90: Conv2d(160, 960, 1, 1, 0, 1)
91: ReLU6
92: Conv2d(1, 960, 3, 1, 1, 960)
93: ReLU6
94: Conv2d(960, 320, 1, 1, 0, 1)
95: Conv2d(320, 1280, 1, 1, 0, 1)
96: ReLU6
97: GlobalAveragePool
98: Linear(1280, 1000)
back 5 1#

```

Input	Operator	t	Output Channels	Repeat	Stride
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1×1	-	1280	1	1
$7^2 \times 1280$	avgpool 7×7	-	-	1	-